เอกสารประกอบการสอน

วิชา โครงสรางระบบคอมพิวเตอร์

(Computer System Organization)

บทที่ 4 ไมโครโปรเซสเซอร์และการประมวลผล

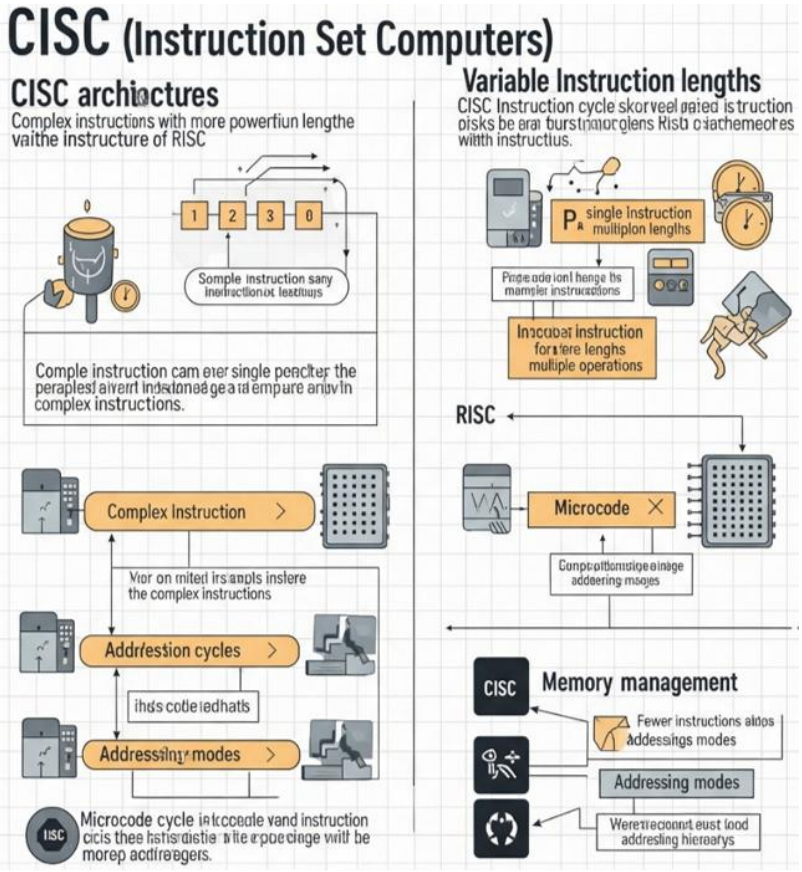
(Microprocessor and Process)

วัตถุประสงค์เชิงพฤติกรรม (Behavioral Objectives)

1. อธิบายโครงสร้างสถาปัตยกรรมคอมพิวเตอร์แบบ CISC (Complex Instruction Set Computer) ได้
2. อธิบายโครงสร้างสถาปัตยกรรมคอมพิวเตอร์แบบ RISC (Reduced Instruction Set Computer) ได้
3. อธิบายการเปรียบเทียบโครงสร้างและการทำงานของสถาปัตยกรรม CISC และ RISC ได้
4. อธิบายไมโครโปรเซสเซอร์ที่ใช้สถาปัตยกรรม CISC และ RISC ได้ใช้งานในปัจจุบัน
5. อธิบายส่วนประกอบโครงสร้างพื้นฐานและโครงสร้างภายในของไมโครโปรเซลเซอร์ได้
6. อธิบายหน้าที่และการทำงานของ Register ได้
7. อธิบายหน้าที่และการทำงานประเภท User-Visible Registers แบบ General Purpose Register (รีจิสเตอร์อเนกประสงค์) ได้
8. อธิบายหน้าที่และการทำงานประเภท User-Visible Registers แบบ Data Register, Address Register (รีจิสเตอร์ข้อมูล) ได้
9. อธิบายหน้าที่การทำงานประเภท User-Visible Registers แบบ Address Registers (รีจิสเตอร์ที่อยู่) ได้
10. อธิบายหน้าที่และการทำงานประเภท User-Visible Registers แบบ Index Registers (รีจิสเตอร์ดัชนี) ได้
11. อธิบายหน้าที่และการทำงานประเภท User-Visible Registers แบบ Segment Registers (รีจิสเตอร์เซกเมนต์) ได้
12. อธิบายหน้าที่และการทำงานประเภท User-Visible Registers แบบ Stack Pointer (SP) (รีจิสเตอร์ตัวชี้สแตก) ได้
13. อธิบายหน้าที่และการทำงานประเภท Condition Codes / Flags Register (รีจิสเตอร์รหัสเงื่อนไข / แฟล็ก) ได้
14. อธิบายหน้าที่การทำงานประเภท Control and Status Registers แบบ Program Counter (PC) ได้
15. อธิบายหน้าที่การทำงานประเภท Control and Status Registers แบบ Instruction Register (IR) ได้
16. อธิบายหน้าที่และการทำงานประเภท Control and Status Registers แบบ Memory Address Register (MAR) ได้
17. อธิบายหน้าที่และการทำงานประเภท Control and Status Registers แบบ Memory Buffer Register (MBR) ได้
18. อธิบายหน้าที่และการทำงานประเภท Control and Status Registers แบบ Program Status Word (PSW) ได้
19. อธิบายการบริหารจัดการองค์ประกอบของ User-Visible Registers และ Control and Status Registers ได้
20. บอกการทำงาน วงรอบคำสั่ง (Interrupt Cycle) และการทำงานของ Interrupt ได้

21. อธิบายวัตถุประสงค์หลักของการเกิด Interrupt ได้
22. อธิบายกระบวนการ Interrupt Cycle (วัฏจักรอินเทอร์รัปต์) ได้
23. ฮธิบายการทำงานของวงรอบคำสั่งของ CPU (CPU Instruction Cycle) ปกติได้
24. อธิบายวงรอบคำสั่งของ CPU พร้อมแทรกการทำงานของ Interrupt ได้
25.. อธิบายกระบวนการ INSTRUCTION CYCLE WITH INDIRECT ได้
26. อธิบายกระบวนการ DATA FLOW (FETCH DIAGRAM) ได้
27. อธิบายกระบวนการ DATA FLOW (INDIRECT DIAGRAM) ได้

1.โครงสร้างสถาปัตยกรรมคอมพิวเตอร์แบบ CISC (Complex Instruction Set Computer)



รูปที่ 4-1 คุณลักษณะและโครงสร้างการทำงานของสถาปัตยกรรมชุดคำสั่งแบบซับซ้อน (CISC Architecture Characteristics)

สถาปัตยกรรม CISC (Complex Instruction Set Computer) เป็นแนวคิดในการออกแบบหน่วยประมวลผลกลาง (CPU) ที่เน้นการมีชุดคำสั่งที่ซับซ้อนและหลากหลาย โดยมีลักษณะสำคัญดังนี้:

1. **ชุดคำสั่งซับซ้อน (Complex Instructions):**
    - คำสั่งแต่ละคำสั่งใน CISC สามารถทำงานได้หลายขั้นตอนในคำสั่งเดียว (multi-cycle operations) คล้ายกับการรวมหลายๆ คำสั่งย่อยเข้าไว้ด้วยกัน
    - ตัวอย่างเช่น คำสั่งเดียวอาจสามารถโหลดข้อมูลจากหน่วยความจำ, ทำการคำนวณ, และเก็บผลลัพธ์กลับไปยังหน่วยความจำได้ในครั้งเดียว
    - สิ่งนี้ช่วยลดจำนวนคำสั่งที่ต้องใช้ในการเขียนโปรแกรม ทำให้โค้ดสั้นลง
2. **ความยาวของคำสั่งไม่คงที่ (Variable-Length Instructions):**
    - เนื่องจากความซับซ้อนของคำสั่งแต่ละคำสั่ง ทำให้ความยาวของรหัสคำสั่ง (Instruction Code) ไม่เท่ากัน
    - บางคำสั่งอาจใช้เพียงไม่กี่ไบต์ ในขณะที่บางคำสั่งอาจใช้หลายไบต์ ขึ้นอยู่กับความซับซ้อนของการทำงานและจำนวนโอเปอแรนด์ (operands) ที่เกี่ยวข้อง

3. **การใช้ Microcode (ไมโครโค้ด):**
    - การทำงานของคำสั่งที่ซับซ้อนใน CISC มักถูกควบคุมโดย "ไมโครโค้ด" (Microcode) ซึ่งเป็นโปรแกรมขนาดเล็กที่เก็บอยู่ใน ROM ภายใน CPU
    - เมื่อ CPU รับคำสั่งที่ซับซ้อนเข้ามา หน่วยควบคุม (Control Unit) จะไปเรียกใช้ลำดับของไมโครโค้ดที่เกี่ยวข้อง เพื่อแปลงคำสั่งซับซ้อนนั้นให้กลายเป็นชุดของ "ไมโครโอเปอเรชัน" (Micro-operations) ที่เรียบง่ายกว่า ซึ่ง CPU สามารถดำเนินการได้โดยตรง
    - การใช้ไมโครโค้ดทำให้การออกแบบฮาร์ดแวร์ของ CPU มีความยืดหยุ่นมากขึ้น และสามารถเพิ่มคำสั่งใหม่ๆ ได้ง่ายขึ้นโดยไม่ต้องเปลี่ยนโครงสร้างฮาร์ดแวร์ทั้งหมด
4. **การเข้าถึงหน่วยความจำโดยตรง (Direct Memory Access):**
    - คำสั่งใน CISC สามารถเข้าถึงหน่วยความจำได้โดยตรงในหลายๆ รูปแบบ (Addressing Modes) ทำให้การจัดการข้อมูลในหน่วยความจำมีความยืดหยุ่นสูง
    - คำสั่งบางคำสั่งสามารถดำเนินการกับข้อมูลที่อยู่ในหน่วยความจำได้โดยตรง โดยไม่ต้องโหลดข้อมูลเข้าสู่รีจิสเตอร์ก่อน ซึ่งแตกต่างจาก RISC ที่มักจะทำงานกับรีจิสเตอร์เป็นหลัก
5. **จำนวนรีจิสเตอร์น้อยกว่า (Fewer Registers):**
    - สถาปัตยกรรม CISC มักจะมีจำนวนรีจิสเตอร์ภายใน CPU น้อยกว่าเมื่อเทียบกับ RISC
    - เนื่องจากคำสั่งสามารถทำงานกับหน่วยความจำได้โดยตรง ทำให้ไม่จำเป็นต้องมีรีจิสเตอร์จำนวนมากเพื่อเก็บข้อมูลชั่วคราว

ข้อดีของ CISC:

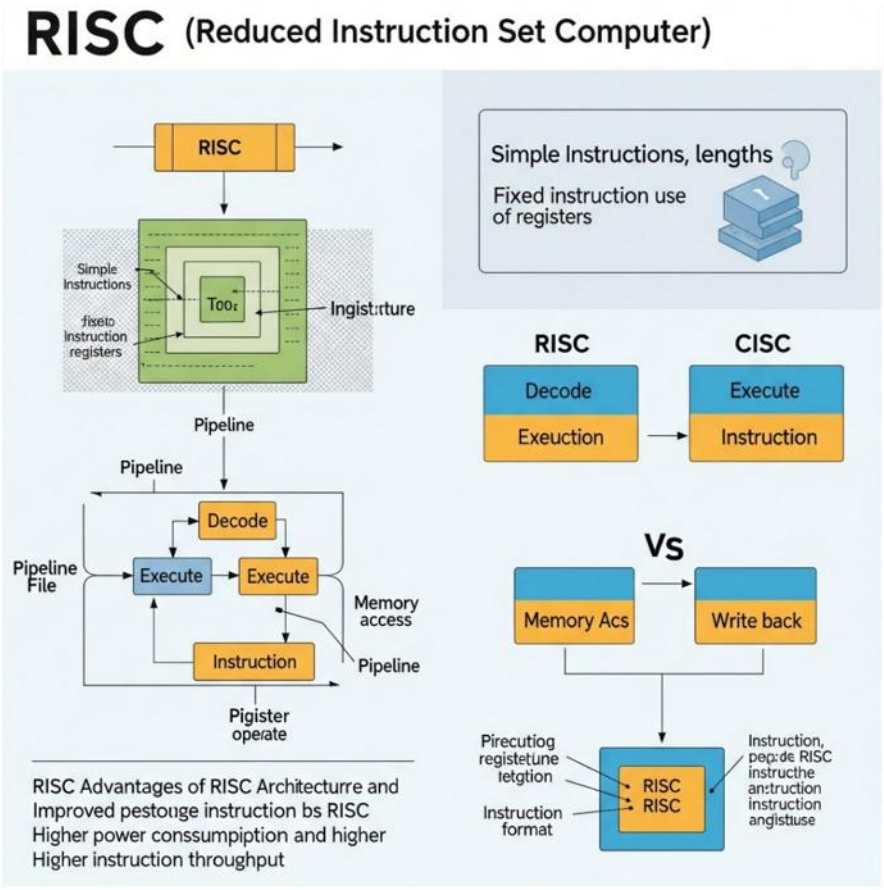
- **โค้ดสั้นลง:** คำสั่งเดียวทำงานได้หลายอย่าง ทำให้โปรแกรมที่เขียนด้วยภาษา Assembly สั้นลง
- **ลดภาระ Compiler:** เนื่องจากคำสั่งมีความซับซ้อนและใกล้เคียงกับการทำงานระดับสูง ทำให้ Compiler (ตัวแปลภาษาโปรแกรม) มีภาระในการแปลงโค้ดน้อยลง

ข้อเสียของ CISC:

- **ความเร็วในการประมวลผลต่อคำสั่ง:** คำสั่งที่ซับซ้อนอาจต้องใช้หลาย Clock Cycle ในการประมวลผล ทำให้ความเร็วโดยรวมช้าลงในบางกรณี
- **การออกแบบที่ซับซ้อน:** การออกแบบ CPU มีชุดคำสั่งซับซ้อนทำได้ยากและใช้ทรัพยากรมาก
- **ยากต่อการทำ Pipelining:** การที่คำสั่งมีความยาวไม่เท่ากันและทำงานหลายขั้นตอน ทำให้การทำ Pipelining (การประมวลผลคำสั่งแบบขนาน) ทำได้ยากและไม่มีประสิทธิภาพเท่าที่ควร

ตัวอย่างไมโครโปรเซสเซอร์ที่ใช้สถาปัตยกรรม CISC: *ไมโครโปรเซสเซอร์ตระกูล Intel x86 (เช่น Pentium, Celeron, Xeon) และ Motorola 68XXX เป็นตัวอย่างที่โดดเด่นของสถาปัตยกรรม CISC ที่ใช้งานอย่างแพร่หลายในคอมพิวเตอร์ส่วนบุคคลมาอย่างยาวนาน*

2.โครงสร้างสถาปัตยกรรมคอมพิวเตอร์แบบ RISC (Reduced Instruction Set Computer)



รูปที่ 4-2 การเปรียบเทียบโครงสร้างและกระบวนการประมวลผลระหว่างสถาปัตยกรรม CISC และ RISC

สถาปัตยกรรม RISC เป็นแนวคิดในการออกแบบหน่วยประมวลผลกลาง (CPU) ที่เน้นความเรียบง่ายของชุดคำสั่ง เพื่อให้สามารถประมวลผลได้รวดเร็วและมีประสิทธิภาพสูง โดยมีลักษณะสำคัญดังนี้:

1. **ชุดคำสั่งเรียบง่ายและน้อยลง (Reduced and Simple Instruction Set):**

   คำสั่งใน RISC จะถูกออกแบบให้ทำงานเพียงหนึ่งขั้นตอนพื้นฐานต่อคำสั่ง (single-cycle operations)

   แต่ละคำสั่งจะทำหน้าที่เฉพาะเจาะจงและไม่ซับซ้อน เช่น คำสั่งสำหรับการโหลดข้อมูล (Load), การจัดเก็บข้อมูล (Store), การคำนวณทางคณิตศาสตร์ (Arithmetic), หรือการดำเนินการทางตรรกะ (Logical) เท่านั้น

   - การทำงานที่ซับซ้อนจะต้องประกอบจากหลายๆ คำสั่งพื้นฐานเข้าด้วยกัน

2. **ความยาวของคำสั่งคงที่ (Fixed-Length Instructions):**
   - คำสั่งทุกคำสั่งใน RISC จะมีความยาวของรหัสคำสั่งเท่ากันทั้งหมด
   - สิ่งนี้ช่วยให้ CPU สามารถดึง (Fetch) และถอดรหัส (Decode) คำสั่งได้อย่างรวดเร็วและสม่ำเสมอ ซึ่งเป็นหัวใจสำคัญของการทำ Pipelining

3. **การใช้รีจิสเตอร์จำนวนมาก (Extensive Use of Registers):**
    - RISC CPU มักจะมีจำนวนรีจิสเตอร์ภายในเป็นจำนวนมาก (เช่น 32, 64 หรือมากกว่า)
    - การทำงานส่วนใหญ่จะเกิดขึ้นภายในรีจิสเตอร์ โดยข้อมูลจะถูกโหลดจากหน่วยความจำเข้าสู่รีจิสเตอร์ก่อนที่จะถูกประมวลผล และผลลัพธ์จะถูกเก็บในรีจิสเตอร์ก่อนที่จะถูกเขียนกลับไปยังหน่วยความจำ (Load/Store Architecture)
    - การใช้รีจิสเตอร์จำนวนมากช่วยลดการเข้าถึงหน่วยความจำ ซึ่งเป็นคอขวดที่ทำให้การประมวลผลช้าลง
4. **เน้นการทำงานแบบ Pipelining (Pipelining Emphasis):**
    - เนื่องจากคำสั่งมีความเรียบง่ายและมีความยาวคงที่ ทำให้ RISC เหมาะสมอย่างยิ่งกับการทำ Pipelining
    - Pipelining คือเทคนิคที่ทำให้ CPU สามารถประมวลผลหลายๆ คำสั่งพร้อมกันในแต่ละขั้นตอนของวงรอบคำสั่ง (เช่น ในขณะที่คำสั่งหนึ่งกำลังถูก Execute, อีกคำสั่งหนึ่งกำลังถูก Decode และอีกคำสั่งหนึ่งกำลังถูก Fetch) ซึ่งช่วยเพิ่มประสิทธิภาพการทำงานโดยรวมได้อย่างมาก
5. **การควบคุมด้วยฮาร์ดแวร์ (Hardwired Control):**
    - หน่วยควบคุม (Control Unit) ใน RISC มักจะถูกออกแบบด้วยฮาร์ดแวร์โดยตรง (Hardwired Control) แทนที่จะใช้ไมโครโค้ด
    - การควบคุมด้วยฮาร์ดแวร์ทำให้การทำงานของแต่ละคำสั่งรวดเร็วและมีประสิทธิภาพสูงกว่า แต่ก็ทำให้การเพิ่มคำสั่งใหม่ๆ ทำได้ยากกว่า CISC

ข้อดีของ RISC:

- **ความเร็วสูง:** การประมวลผลคำสั่งที่เรียบง่ายและ Pipelining ที่มีประสิทธิภาพ ทำให้ RISC มีความเร็วในการประมวลผลสูง
- **ออกแบบง่ายกว่า:** การออกแบบ CPU ที่มีชุดคำสั่งเรียบง่ายทำได้ง่ายกว่าและใช้ทรัพยากรน้อยกว่า
- **ประหยัดพลังงาน:** มักจะใช้พลังงานน้อยกว่า ทำให้เหมาะสำหรับอุปกรณ์พกพา

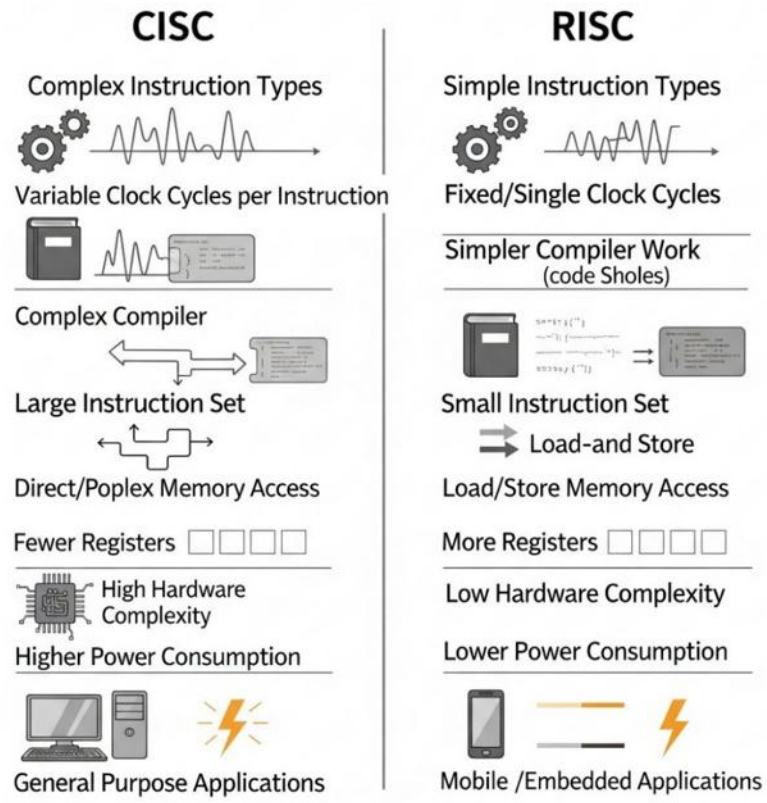
ข้อเสียของ RISC:

- **โค้ดโปรแกรมยาวขึ้น:** การทำงานที่ซับซ้อนต้องใช้หลายคำสั่งพื้นฐาน ทำให้โปรแกรมมีขนาดใหญ่ขึ้น
- **ภาระของ Compiler มากขึ้น:** Compiler ต้องทำงานหนักขึ้นในการแปลงภาษาโปรแกรมระดับสูงให้เป็นชุดคำสั่ง RISC ที่มีประสิทธิภาพ

ตัวอย่างไมโครโปรเซสเซอร์ที่ใช้สถาปัตยกรรม RISC: *ไมโครโปรเซสเซอร์ตระกูล ARM (ที่ใช้ในสมาร์ทโฟนและแท็บเล็ตส่วนใหญ่), PowerPC, SPARC, และ MIPS เป็นตัวอย่างที่โดดเด่นของสถาปัตยกรรม RISC ที่ประสบความสำเร็จอย่างสูงในปัจจุบัน*

3.การเปรียบเทียบโครงสร้างและการทำงานของสถาปัตยกรรม CISC และ RISC

1. **ประเภทคำสั่ง (Instruction Types):**
   - CISC: มีชุดคำสั่งที่ **ซับซ้อน (Complex)** คำสั่งเดียวสามารถทำงานได้หลายขั้นตอน (เช่น โหลดข้อมูล, คำนวณ, และจัดเก็บผลลัพธ์ในคำสั่งเดียว)
   - RISC: มีชุดคำสั่งที่ **เรียบง่าย (Simple)** และไม่ซับซ้อน แต่ละคำสั่งทำงานเพียงหนึ่งขั้นตอนพื้นฐาน (เช่น โหลด, จัดเก็บ, หรือคำนวณ)
2. Clock Cycle per Instruction:
   - CISC: โดยทั่วไป คำสั่งหนึ่งคำสั่งอาจต้องใช้ **หลาย** Clock Cycle ในการประมวลผลให้เสร็จสมบูรณ์ เนื่องจากความซับซ้อนของคำสั่งนั้นๆ
   - RISC: คำสั่งส่วนใหญ่ถูกออกแบบมาให้ประมวลผลเสร็จสิ้นได้ภายใน **หนึ่ง** Clock Cycle ซึ่งเป็นเป้าหมายหลักของการออกแบบ RISC



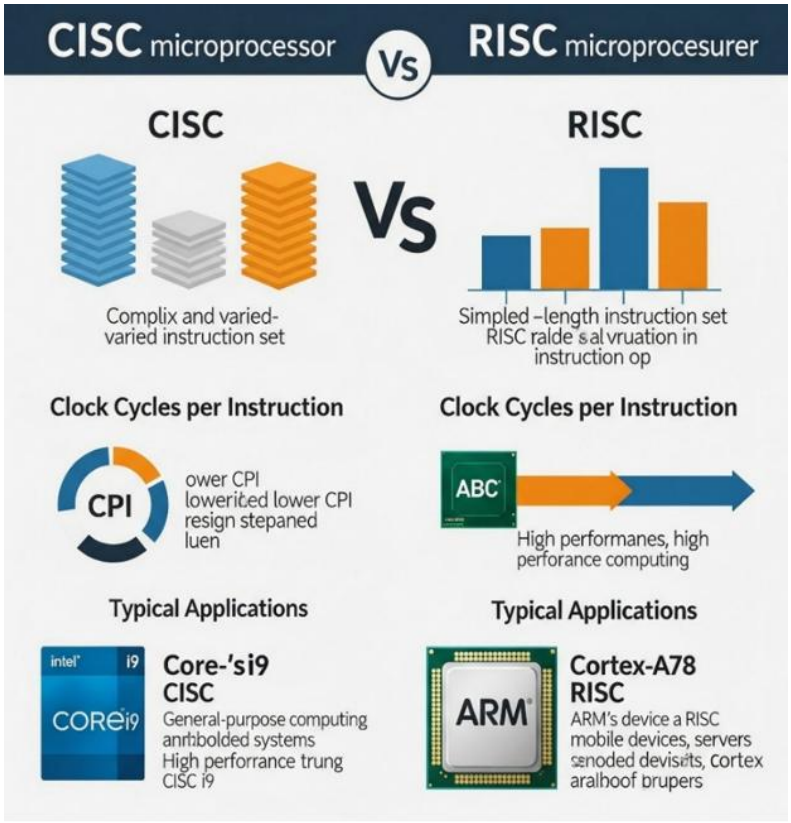
รูปที่ 4-3 สรุปข้อแตกต่างระหว่างสถาปัตยกรรมแบบ CISC และ RISC (Comparison Summary of CISC and RISC)

3. Compiler:
   - CISC: Compiler **มีภาระน้อยกว่า** ในการแปลงโค้ดจากภาษาระดับสูง เนื่องจากมีคำสั่งที่ซับซ้อนคล้ายกับฟังก์ชันในภาษาระดับสูงอยู่แล้ว
   - RISC: Compiler **มีภาระมากกว่า** ในการปรับแต่งโค้ดให้มีประสิทธิภาพสูงสุด เพราะต้องแปลงฟังก์ชันที่ซับซ้อนให้เป็นชุดของคำสั่ง RISC ที่เรียบง่ายหลายๆ คำสั่ง และต้องจัดการเรื่อง Pipelining และการใช้รีจิสเตอร์อย่างมีประสิทธิภาพ
4. **Instruction Set Size (ขนาดของชุดคำสั่ง):**
   - CISC: มี **ชุดคำสั่งขนาดใหญ่ (Large Instruction Set)** มีคำสั่งจำนวนมากและหลากหลายรูปแบบ
   - RISC: มี **ชุดคำสั่งขนาดเล็ก (Small Instruction Set)** มีคำสั่งจำนวนน้อยกว่าและเป็นพื้นฐาน

5. **Memory Access (การเข้าถึงหน่วยความจำ):**
    - **CISC:** สามารถเข้าถึงหน่วยความจำได้ **โดยตรงจากคำสั่ง** (Direct Memory Access in instructions) คำสั่งหลายคำสั่งสามารถดำเนินการกับข้อมูลในหน่วยความจำได้เลย
    - **RISC:** ส่วนใหญ่ใช้แบบ **Load/Store Architecture** คือข้อมูลจะต้องถูกโหลดจากหน่วยความจำเข้าสู่รีจิสเตอร์ก่อนที่จะทำการประมวลผล และผลลัพธ์จะต้องถูกเก็บในรีจิสเตอร์ก่อนที่จะเขียนกลับไปยังหน่วยความจำ
6. **Register Set (ชุดรีจิสเตอร์):**
    - **CISC:** มักมี **จำนวนรีจิสเตอร์น้อยกว่า** (Fewer Registers) เนื่องจากสามารถทำงานกับหน่วยความจำได้โดยตรง
    - **RISC:** มี **จำนวนรีจิสเตอร์มากกว่า** (More Registers) เพื่อรองรับการทำงานส่วนใหญ่ที่เกิดขึ้นภายในรีจิสเตอร์ และช่วยให้ Pipelining มีประสิทธิภาพ
7. **Hardware Complexity (ความซับซ้อนของฮาร์ดแวร์):**
    - **CISC:** มี **หน่วยควบคุมที่ซับซ้อน (Complex Control Unit)** มักใช้ Microcode ในการจัดการคำสั่งที่หลากหลายและซับซ้อน ทำให้การออกแบบฮาร์ดแวร์ซับซ้อนกว่า
    - **RISC:** มี **หน่วยควบคุมที่เรียบง่ายกว่า (Simpler Control Unit)** มักใช้ Hardwired Control ทำให้การออกแบบฮาร์ดแวร์ง่ายกว่า
8. **Power Consumption (การใช้พลังงาน):**
    - **CISC:** โดยทั่วไปมี **การใช้พลังงานสูงกว่า** เนื่องจากความซับซ้อนของโครงสร้างและการทำงาน
    - **RISC:** โดยทั่วไปมี **การใช้พลังงานต่ำกว่า** เนื่องจากความเรียบง่ายและประสิทธิภาพในการทำงานต่อ Clock Cycle ทำให้เหมาะกับอุปกรณ์พกพา
9. **Applications (การประยุกต์ใช้งาน):**
    - **CISC:** มักพบใน **คอมพิวเตอร์ส่วนบุคคลและเซิร์ฟเวอร์** (เช่น Intel x86 Processors) ที่ต้องการความเข้ากันได้กับซอฟต์แวร์เก่าและสามารถจัดการคำสั่งที่ซับซ้อนได้
    - **RISC:** มักพบใน **อุปกรณ์พกพา (Mobile Devices), ระบบสมองกลฝังตัว (Embedded Systems), และ Supercomputers** (เช่น ARM Processors) ที่ต้องการประสิทธิภาพสูง, การประหยัดพลังงาน และการทำงานแบบ Pipelining ที่มีประสิทธิภาพ

4.สถาปัตยกรรมไมโครโปรเซสเซอร์ที่ใช้สถาปัตยกรรม CISC และ RISC ได้ใช้งานในปัจจุบัน

CISC (Complex Instruction Set Computer)



รูปที่ 4-4 การเปรียบเทียบรอบสัญญาณนาฬิกาและตัวอย่างการประยุกต์ใช้งานจริงของโปรเซสเซอร์แบบ CISC และ RISC

CISC เป็นสถาปัตยกรรมที่เน้นการมีชุดคำสั่งที่ซับซ้อนและหลากหลาย โดยแต่ละคำสั่งสามารถทำงานได้หลายขั้นตอนในครั้งเดียว คล้ายกับการที่อาจารย์สั่งงานนักศึกษาด้วยประโยคยาวๆ ที่ครอบคลุมหลายๆ อย่างในคำสั่งเดียว

ลักษณะเด่นของ CISC:

- **ชุดคำสั่งซับซ้อน:** มีคำสั่งจำนวนมากและแต่ละคำสั่งสามารถทำงานได้หลายอย่าง (เช่น การโหลดข้อมูล, การคำนวณ, และการจัดเก็บข้อมูล) ในคำสั่งเดียว
- **คำสั่งความยาวไม่คงที่:** ความยาวของคำสั่งแต่ละคำสั่งอาจไม่เท่ากัน ทำให้การถอดรหัสคำสั่งซับซ้อน
- **ใช้ Clock Cycle มากกว่าต่อคำสั่ง:** เนื่องจากแต่ละคำสั่งทำงานหลายขั้นตอน จึงอาจใช้เวลาหลาย Clock Cycle ในการประมวลผลหนึ่งคำสั่ง
- **การเขียนโปรแกรมง่ายขึ้น:** โปรแกรมเมอร์สามารถใช้คำสั่งที่ซับซ้อนเพื่อทำงานที่ต้องการได้โดยใช้โค้ดที่สั้นลง

ตัวอย่างไมโครโปรเซสเซอร์ CISC ที่ใช้งานในปัจจุบัน:

- ***Intel 80x86 family (เช่น 8086, 8088, x86-64bit):*** *เป็นสถาปัตยกรรมที่โดดเด่นและแพร่หลายที่สุดในโลกคอมพิวเตอร์ส่วนบุคคลและเซิร์ฟเวอร์ รวมถึงซีพียูที่เราใช้กันในคอมพิวเตอร์ตั้งโต๊ะและโน้ตบุ๊กส่วนใหญ่ เช่น Intel Core i3, i5, i7, i9*
- ***Motorola 68XXX family:*** *เคยเป็นที่นิยมในคอมพิวเตอร์ Macintosh ยุคแรกๆ และระบบฝังตัวบางชนิด*
- ***Pentium, Celeron, Xeon, Atom:*** *ทั้งหมดนี้เป็นชื่อรุ่นของไมโครโปรเซสเซอร์จาก Intel ซึ่งล้วนใช้สถาปัตยกรรม x86 ที่เป็น CISC*

4.1 สถาปัตยกรรม RISC (Reduced Instruction Set Computer)

RISC เป็นสถาปัตยกรรมที่มีชุดคำสั่งที่เรียบง่ายและลดจำนวนคำสั่งลง โดยแต่ละคำสั่งจะทำงานเพียงขั้นตอนเดียวเท่านั้น คล้ายกับการที่อาจารย์สั่งงานนักศึกษาด้วยประโยคสั้นๆ ที่เจาะจงให้ทำทีละอย่าง

ลักษณะเด่นของ RISC:

- **ชุดคำสั่งเรียบง่าย:** มีคำสั่งจำนวนน้อยลงและแต่ละคำสั่งทำงานเพียงหนึ่งขั้นตอนเท่านั้น (เช่น โหลดข้อมูลอย่างเดียว, คำนวณอย่างเดียว, หรือจัดเก็บข้อมูลอย่างเดียว)
- **คำสั่งความยาวคงที่:** ความยาวของคำสั่งแต่ละคำสั่งเท่ากัน ทำให้การถอดรหัสคำสั่งง่ายและรวดเร็ว
- **ใช้ Clock Cycle น้อยกว่าต่อคำสั่ง:** มักจะใช้เพียงหนึ่ง Clock Cycle ในการประมวลผลหนึ่งคำสั่ง ทำให้การทำงานรวดเร็วและมีประสิทธิภาพสูง
- **การออกแบบง่ายกว่า:** การออกแบบฮาร์ดแวร์ของทำได้ง่ายกว่า ทำให้ประหยัดพลังงานและต้นทุนการผลิตต่ำ

ตัวอย่างไมโครโปรเซสเซอร์ RISC ที่ใช้งานในปัจจุบัน:

- ***PowerPC:*** *พัฒนาโดยความร่วมมือระหว่าง Apple, IBM, และ Motorola เคยใช้ในคอมพิวเตอร์ Apple Macintosh ในระบบฝังตัว, คอนโซลเกม (เช่น Xbox 360, PlayStation 3) และซูเปอร์คอมพิวเตอร์*
- ***MIPS (Microprocessor without Interlocked Pipeline Stages):*** *ใช้กันอย่างแพร่หลายในอุปกรณ์เครือข่าย, ระบบฝังตัว, และคอนโซลเกมบางชนิด (เช่น Nintendo 64, PlayStation)*
- ***ARM (Advanced RISC Machine):*** *เป็นสถาปัตยกรรม RISC ที่ได้รับความนิยมอย่างมหาศาลและแพร่หลายที่สุดในอุปกรณ์พกพา เช่น สมาร์ทโฟน, แท็บเล็ต, อุปกรณ์ IoT, และคอมพิวเตอร์ส่วนบุคคลบางรุ่น (เช่น Apple Silicon ใน Mac)*

คุณสมบัติ	CISC	RISC
ชุดคำสั่ง	ซับซ้อน, หลากหลาย, ทำงานได้หลายขั้นตอน	เรียบง่าย, จำนวนน้อย, ทำงานทีละขั้นตอนเดียว
ความยาวคำสั่ง	ไม่คงที่	คงที่
Clock Cycle/คำสั่ง	หลาย Clock Cycle	โดยทั่วไป 1 Clock Cycle
ความซับซ้อนของ Hardware	สูง	ต่ำ
การใช้พลังงาน	สูงกว่า	ต่ำกว่า
การเขียนโปรแกรม	ง่ายกว่า (โค้ดสั้น)	ซับซ้อนกว่า (โค้ดยาวขึ้น)
การใช้งานหลัก	คอมพิวเตอร์ส่วนบุคคล, เซิร์ฟเวอร์ทั่วไป	อุปกรณ์พกพา, ระบบฝังตัว, เซิร์ฟเวอร์ประสิทธิภาพสูง, ซูเปอร์คอมพิวเตอร์

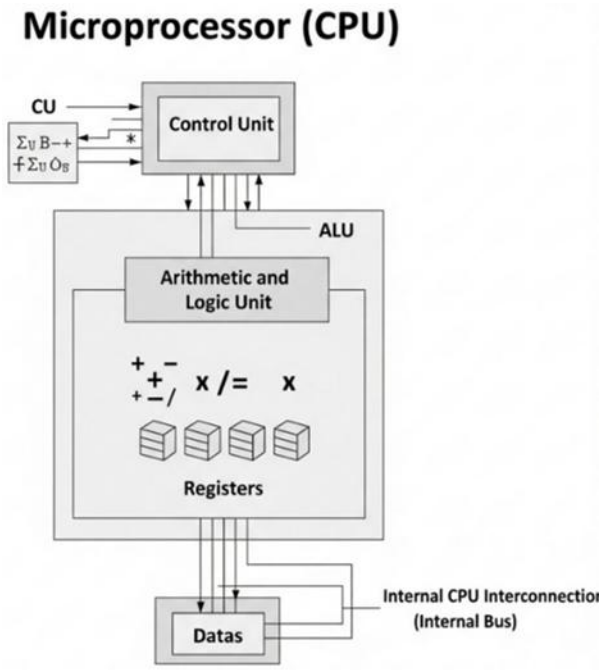
ตารางที่ 4-1 การเปรียบเทียบคุณสมบัติและข้อแตกต่างระหว่างสถาปัตยกรรม CISC และ RISC

5.ส่วนประกอบโครงสร้างพื้นฐานและโครงสร้างภายในของไมโครโปรเซลเซอร์

1.โครงสร้างโปรเซสเซอร์ (Processor Organization): แผนภาพบล็อกหลักของ CPU

ไมโครโปรเซสเซอร์ หรือ CPU (Central Processing Unit) เปรียบเสมือนสมองของคอมพิวเตอร์ ทำหน้าที่ประมวลผลคำสั่งและข้อมูลต่างๆ โครงสร้างพื้นฐานของ CPU ประกอบด้วยส่วนหลักๆ ดังนี้:

- **หน่วยควบคุม (Control Unit - CU):** ทำหน้าที่ควบคุมประสานงานการทำงานของทุกส่วนใน CPU
- **หน่วยคำนวณและตรรกะ (Arithmetic and Logic Unit - ALU):** ทำหน้าที่ประมวลผลทางคณิตศาสตร์และตรรกะ
- **รีจิสเตอร์ (Registers):** ทำหน้าที่เก็บข้อมูลชั่วคราวที่ CPU กำลังประมวลผลอยู่
- **ระบบเชื่อมโยงภายใน CPU (Internal CPU Interconnection):** หรือที่เรียกว่า Internal Bus ทำหน้าที่เป็นเส้นทางสื่อสารข้อมูลระหว่างส่วนประกอบต่างๆ ภายใน CPU



รูปที่ 4-5 ส่วนประกอบหลักและโครงสร้างภายในของไมโครโปรเซสเซอร์

2. หน้าที่ของหน่วยควบคุม (Control Unit - CU)

หน่วยควบคุม (CU) เป็นส่วนที่สำคัญที่สุดในการบริหารจัดการการทำงานทั้งหมดของ CPU หน้าที่หลักๆ ของ CU ได้แก่:

- **การถอดรหัสคำสั่ง (Instruction Decoding):** เมื่อ CPU ดึงคำสั่งมาจากหน่วยความจำ CU จะทำหน้าที่ถอดรหัสคำสั่งนั้นเพื่อทำความเข้าใจว่าต้องทำอะไร
- **การควบคุมลำดับการทำงาน (Sequencing):** CU จะกำหนดลำดับขั้นตอนการทำงานของ CPU ให้ถูกต้องตามคำสั่งที่ได้รับมา
- **การสร้างสัญญาณควบคุม (Control Signal Generation):** CU จะสร้างสัญญาณไฟฟ้าควบคุม (Control Signals) เพื่อสั่งการให้ส่วนประกอบอื่นๆ ของ CPU เช่น ALU, Registers, หรือหน่วยความจำภายนอก ทำงานตามที่ต้องการ ตัวอย่างเช่น สัญญาณอ่าน (Read), สัญญาณเขียน (Write), สัญญาณโหลด (Load), สัญญาณจัดเก็บ (Store) เป็นต้น
- **การจัดการการไหลของข้อมูล (Data Flow Management):** CU จะควบคุมทิศทางการไหลของข้อมูลระหว่างส่วนประกอบต่างๆ ภายใน CPU และระหว่าง CPU กับอุปกรณ์ภายนอก

ตัวอย่าง: *เมื่อนักศึกษาสั่งให้คอมพิวเตอร์เปิดโปรแกรม CU จะเป็นผู้รับคำสั่งนี้ ถอดรหัส และส่งสัญญาณควบคุมไปยังส่วนต่างๆ เพื่อโหลดโปรแกรมจากฮาร์ดดิสก์เข้าสู่หน่วยความจำ และเริ่มการทำงานของโปรแกรม*

3. หน้าที่ของหน่วยคำนวณและตรรกะ (ALU): เป็นส่วนที่ทำหน้าที่หลักในการประมวลผลข้อมูล

- **การประมวลผลทางคณิตศาสตร์ (Arithmetic Operations):** ทำหน้าที่คำนวณทางคณิตศาสตร์พื้นฐาน เช่น การบวก (Addition), การลบ (Subtraction), การคูณ (Multiplication), การหาร (Division)
- **การประมวลผลทางตรรกะ (Logic Operations):** ทำหน้าที่เปรียบเทียบข้อมูลและดำเนินการทางตรรกะ เช่น AND, OR, NOT, XOR รวมถึงการเปรียบเทียบค่า (เช่น น้อยกว่า, มากกว่า, เท่ากับ)
- **การเลื่อนบิต (Shift Operations):** การเลื่อนบิตข้อมูลไปทางซ้ายหรือขวา ซึ่งใช้ในการคูณหรือหารด้วยกำลังของสองอย่างรวดเร็ว

ตัวอย่าง: *หากนักศึกษากำลังใช้โปรแกรมคำนวณ (Calculator) เพื่อบวกเลข 5 + 3, ALU จะเป็นส่วนที่รับค่า 5 และ 3 เข้าไป และทำการบวกให้ได้ผลลัพธ์เป็น 8*

4. หน้าที่ของ Registers

รีจิสเตอร์ (Registers) คือหน่วยความจำขนาดเล็กที่มีความเร็วสูงมาก ตั้งอยู่ภายใน CPU เอง ทำหน้าที่เก็บข้อมูลชั่วคราวที่ CPU กำลังใช้งานอยู่ หรือข้อมูลที่กำลังจะถูกประมวลผล หรือผลลัพธ์จากการประมวลผลที่เพิ่งเสร็จสิ้น

ลักษณะสำคัญของ Registers:

- **ความเร็วสูง:** เป็นหน่วยความจำที่เร็วที่สุดในระบบเพราะอยู่ใกล้กับ ALU และ CU มากที่สุด
- **จำนวนจำกัด:** มีจำนวนน้อยเมื่อเทียบกับหน่วยความจำหลัก (RAM)
- **วัตถุประสงค์เฉพาะ:** รีจิสเตอร์บางตัวมีวัตถุประสงค์เฉพาะ เช่น Program Counter (PC) เก็บที่อยู่ของคำสั่งถัดไป, Instruction Register (IR) เก็บคำสั่งที่กำลังถูกประมวลผล, Accumulator (AC) เก็บผลลัพธ์ชั่วคราวจากการคำนวณ

ตัวอย่าง: *เมื่อ ALU ทำการบวกเลข 5 + 3 ได้ผลลัพธ์ 8 ผลลัพธ์ 8 นี้จะถูกเก็บไว้ในรีจิสเตอร์ชั่วคราว ก่อนที่จะถูกส่งไปยังหน่วยความจำหลัก หรือถูกนำไปใช้ในการคำนวณขั้นตอนต่อไป*

5. ระบบเชื่อมโยงภายใน CPU (Internal CPU Interconnection): Internal Bus คือชุดของเส้นทางไฟฟ้าที่ทำหน้าที่เป็นช่องทางการสื่อสารข้อมูลระหว่างส่วนประกอบต่างๆ ภายใน CPU เอง

ประเภทของบัสภายใน CPU:

- **Data Bus:** ใช้สำหรับขนส่งข้อมูลระหว่าง ALU, Registers, และส่วนอื่นๆ
- **Address Bus:** ใช้สำหรับระบุตำแหน่ง (Address) ของข้อมูลในหน่วยความจำหรืออุปกรณ์ I/O
- **Control Bus:** ใช้สำหรับส่งสัญญาณควบคุมจาก CU ไปยังส่วนต่างๆ และรับสัญญาณสถานะกลับมา

ตัวอย่าง: *เมื่อ CU ต้องการให้ ALU ทำการคำนวณ ข้อมูลที่จะคำนวณจะถูกส่งผ่าน Data Bus ไปยัง ALU และผลลัพธ์ก็จะถูกส่งกลับผ่าน Data Bus ไปยัง Registers หรือหน่วยความจำ*

6.การทำงานชอง Register ประเภท User-Visible Register

1. CPU Registers และ หน้าที่ของ Register

CPU Registers คือ หน่วยความจำขนาดเล็กพิเศษที่มีความเร็วสูงมาก ตั้งอยู่ภายในหน่วยประมวลผลกลาง (CPU) โดยตรง รีจิสเตอร์เหล่านี้มีความสำคัญอย่างยิ่งต่อการทำงานของ CPU เพราะเป็นพื้นที่สำหรับเก็บข้อมูลชั่วคราวที่ CPU ต้องการเข้าถึงและประมวลผลอย่างรวดเร็วที่สุด

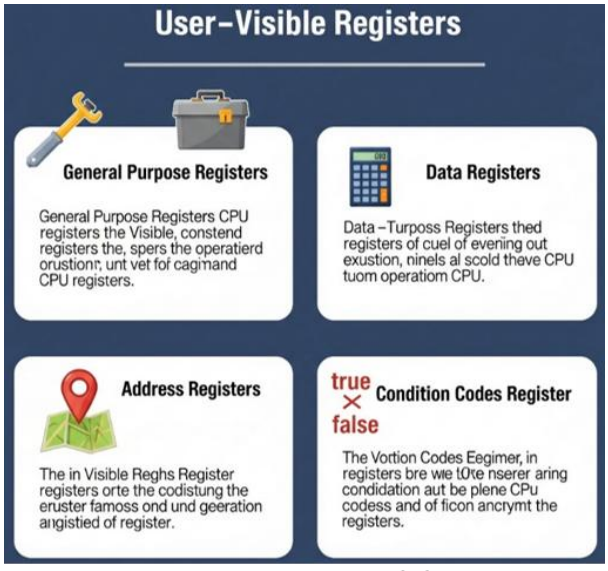
หน้าที่หลักของ Register คือ:

- **เก็บข้อมูลชั่วคราว:** ใช้เก็บข้อมูลที่กำลังถูกประมวลผล (Operands) หรือผลลัพธ์จากการประมวลผลที่เพิ่งเสร็จสิ้น เพื่อให้ ALU สามารถเข้าถึงข้อมูลได้อย่างรวดเร็วโดยไม่ต้องรอการเข้าถึงจากหน่วยความจำหลัก (RAM)
- **เก็บคำสั่ง:** เก็บคำสั่ง (Instructions) ที่กำลังจะถูกประมวลผล
- **เก็บที่อยู่:** เก็บที่อยู่ (Addresses) ของข้อมูลหรือคำสั่งในหน่วยความจำ
- **เก็บสถานะ:** เก็บสถานะของการทำงานของ CPU หรือผลลัพธ์จากการดำเนินการ (เช่น มีการทด, ผลลัพธ์เป็นศูนย์)

ตัวอย่าง การใช้รีจิสเตอร์ช่วยลดเวลาในการเข้าถึงข้อมูลจากหน่วยความจำหลัก ซึ่งมีความเร็วช้ากว่ามาก ทำให้การทำงานของ CPU มีประสิทธิภาพและความเร็วสูงขึ้นอย่างเห็นได้ชัด

2. หน้าที่และการทำงานของ User-Visible Registers

"User-Visible Registers" คือ รีจิสเตอร์ที่โปรแกรมเมอร์ (หรือผู้เขียนโปรแกรมระดับแอสเซมบลี) สามารถเข้าถึงและใช้งานได้โดยตรง เพื่อควบคุมการทำงานหรือเก็บข้อมูลที่จำเป็นในการเขียนโปรแกรมสามารถแบ่งได้ดังนี้:



รูปที่ 4-6 ประเภทของรีจิสเตอร์ที่ผู้ใช้งานสามารถมองเห็นและเรียกใช้งานได้

a) General-Purpose Registers (รีจิสเตอร์อเนกประสงค์)

- **หน้าที่:** เป็นรีจิสเตอร์ที่ยืดหยุ่นที่สุด สามารถใช้เก็บข้อมูลได้หลากหลายประเภท ทั้งข้อมูลตัวเลข, ตัวอักษร, หรือที่อยู่ของหน่วยความจำ ขึ้นอยู่กับความต้องการของโปรแกรมที่กำลังทำงานอยู่
- **การใช้งาน:** โปรแกรมเมอร์สามารถนำไปใช้ในการดำเนินงานต่างๆ เช่น การบวก, ลบ, คูณ, หาร, การย้ายข้อมูล หรือเป็นตัวแปรชั่วคราวในโปรแกรม
- ***ตัวอย่าง:*** *ในสถาปัตยกรรม x86 เช่น AX, BX, CX, DX (ในอดีต) หรือ RAX, RBX, RCX, RDX (ใน 64-bit) และ R8-R15 (ใน 64-bit) เป็นต้น*

b) Data Registers (รีจิสเตอร์ข้อมูล)

- **หน้าที่:** ใช้สำหรับเก็บข้อมูลที่กำลังจะถูกประมวลผลโดย ALU หรือผลลัพธ์ที่ได้จากการประมวลผล
- **ความแตกต่างจาก General-Purpose:** บางสถาปัตยกรรมอาจแยก Data Register ออกมาอย่างชัดเจน เพื่อให้มีประสิทธิภาพสูงสุดในการจัดการข้อมูลที่จะเข้าสู่ ALU โดยเฉพาะ อย่างไรก็ตาม ในหลายสถาปัตยกรรม รีจิสเตอร์อเนกประสงค์ก็มักจะถูกใช้เป็น Data Register ไปในตัว
- ***ตัวอย่าง:*** *ในบาง CPU อาจมีรีจิสเตอร์เฉพาะสำหรับข้อมูลที่ไม่มีการใช้เพื่อการกำหนดแอดเดรส*

c) Address Registers (รีจิสเตอร์ที่อยู่)

- **หน้าที่:** ใช้สำหรับเก็บที่อยู่ของหน่วยความจำ (Memory Address) เพื่อใช้ในการเข้าถึงข้อมูลหรือคำสั่งที่อยู่ในหน่วยความจำหลัก
- **การใช้งาน:** CPU ใช้ Address Register ในการระบุตำแหน่งที่จะอ่านหรือเขียนข้อมูลใน RAM หรือในอุปกรณ์ I/O
- ***ตัวอย่าง:*** *ในบางสถาปัตยกรรมอาจมี Address Register แยกต่างหากจาก Data Register เพื่อเพิ่มประสิทธิภาพในการคำนวณที่อยู่*

d) Index Registers (รีจิสเตอร์ดัชนี)

- **หน้าที่:** ใช้เก็บค่า "ดัชนี" (Index) ซึ่งโดยทั่วไปจะถูกใช้เพื่อการคำนวณที่อยู่ของข้อมูลในอาร์เรย์ (Array) หรือโครงสร้างข้อมูลที่มีลักษณะเป็นลำดับ
- **การใช้งาน:** ค่าใน Index Register จะถูกบวกหรือลบกับค่าฐาน (Base Address) เพื่อหาที่อยู่จริงของข้อมูลที่ต้องการเข้าถึง เหมาะสำหรับการเข้าถึงข้อมูลที่เป็นชุดๆ
- ***ตัวอย่าง:*** *เมื่อนักศึกษาต้องการเข้าถึงสมาชิกตัวที่ N ในอาร์เรย์ โปรแกรมสามารถใช้ค่าเริ่มต้นของอาร์เรย์บวกกับค่าใน Index Register เพื่อหาตำแหน่งของสมาชิกตัวที่ N ได้อย่างรวดเร็ว*

e) Segment Registers (รีจิสเตอร์เซกเมนต์)

- **หน้าที่:** มีบทบาทสำคัญในสถาปัตยกรรมที่ใช้ระบบการจัดการหน่วยความจำแบบแบ่งส่วน (Segmentation) เช่น สถาปัตยกรรม x86 ของ Intel รีจิสเตอร์เหล่านี้จะเก็บ "ค่าเริ่มต้น" (Base Address) ของแต่ละ Segment (ส่วนของหน่วยความจำ)
- **การใช้งาน:** เมื่อโปรแกรมต้องการเข้าถึงหน่วยความจำ CPU จะนำค่าจาก Segment Register มาผสมกับค่า Offset (ค่าระยะห่างจากจุดเริ่มต้นของ Segment) เพื่อคำนวณหาที่อยู่จริง (Physical Address) ในหน่วยความจำหลัก
- ***ตัวอย่าง:*** *Code Segment (CS), Data Segment (DS), Stack Segment (SS) เป็นต้น*

f) Stack Pointer (SP) (รีจิสเตอร์ตัวชี้สแตก)

- **หน้าที่:** Stack Pointer เป็นรีจิสเตอร์ที่เก็บที่อยู่ของ "ยอด" (Top) ของสแตก (Stack) ซึ่งเป็นโครงสร้างข้อมูลแบบ Last-In, First-Out (LIFO)
- **การใช้งาน:** เมื่อมีการ "ผลัก" (Push) ข้อมูลเข้าสแตก ค่าใน Stack Pointer จะถูกปรับลดลง (หรือเพิ่มขึ้น ขึ้นอยู่กับทิศทางการเติบโตของสแตก) เพื่อชี้ไปยังตำแหน่งว่างถัดไป และเมื่อมีการ "ดึง" (Pop) ข้อมูลออกจากสแตก ค่าใน Stack Pointer ก็จะถูกปรับเพิ่มขึ้น (หรือลดลง)
- ***ตัวอย่าง:*** *ใช้ในการจัดการการเรียกใช้ฟังก์ชันย่อย (Subroutine Call), การส่งค่าพารามิเตอร์, และการบันทึกสถานะของโปรแกรมชั่วคราว*

3. Condition Codes / Flags Register (รีจิสเตอร์รหัสเงื่อนไข / แฟล็ก)

Condition Codes Register หรือที่มักเรียกว่า **Flags Register** เป็นรีจิสเตอร์พิเศษที่ไม่ได้ใช้เก็บข้อมูลทั่วไป แต่ใช้เก็บ "สถานะ" หรือ "เงื่อนไข" ที่เกิดขึ้นหลังจากการประมวลผลคำสั่งทางคณิตศาสตร์หรือตรรกะล่าสุดแต่ละบิตในรีจิสเตอร์นี้จะทำหน้าที่เป็น "แฟล็ก" (Flag) ที่บ่งบอกสถานะเฉพาะ

หน้าที่หลักของ Condition Codes Register:

- **บ่งบอกสถานะการดำเนินการ:** บ่งบอกว่าผลลัพธ์ของการดำเนินการล่าสุดมีคุณสมบัติอย่างไร
- **ใช้ในการตัดสินใจและควบคุมโปรแกรม:** โปรแกรมสามารถใช้แฟล็กเหล่านี้ในการตัดสินใจว่าจะทำงานตามเงื่อนไขใดต่อไป โดยเฉพาะในคำสั่งกระโดดแบบมีเงื่อนไข (Conditional Jump/Branch Instructions)

ตัวอย่างแฟล็กที่สำคัญ:

- **Zero Flag (ZF):** ตั้งค่าเป็น 1 ถ้าผลลัพธ์ของการดำเนินการเป็นศูนย์ (0) หากไม่เป็นศูนย์ จะเป็น 0
- **Carry Flag (CF):** ตั้งค่าเป็น 1 ถ้าเกิดการทด (Carry) ออกจากบิตที่มีนัยสำคัญสูงสุดในการดำเนินการบวก หรือเกิดการยืม (Borrow) ในการดำเนินการลบ
- **Sign Flag (SF):** ตั้งค่าเป็น 1 ถ้าผลลัพธ์ของการดำเนินการเป็นค่าลบ (บิตสูงสุดเป็น 1) หากเป็นค่าบวก จะเป็น 0
- **Overflow Flag (OF):** ตั้งค่าเป็น 1 ถ้าผลลัพธ์ของการดำเนินการเกินขอบเขตที่รีจิสเตอร์จะจัดเก็บได้ (เกิด Overflow)
- **Parity Flag (PF):** ตั้งค่าเป็น 1 ถ้าจำนวนบิต 1 ในผลลัพธ์เป็นจำนวนคู่

ตัวอย่างการใช้งาน: *สมมติว่า ALU คำนวณ 5 - 5 ผลลัพธ์คือ 0 แฟล็ก Zero Flag (ZF) จะถูกตั้งเป็น 1 โปรแกรมถัดไปสามารถตรวจสอบค่า ZF นี้ได้ว่า "ถ้า ZF เป็น 1 ให้กระโดดไปทำงานที่ส่วน A" ซึ่งหมายถึง "ถ้าผลลัพธ์เป็นศูนย์ ให้กระโดดไปส่วน A"*

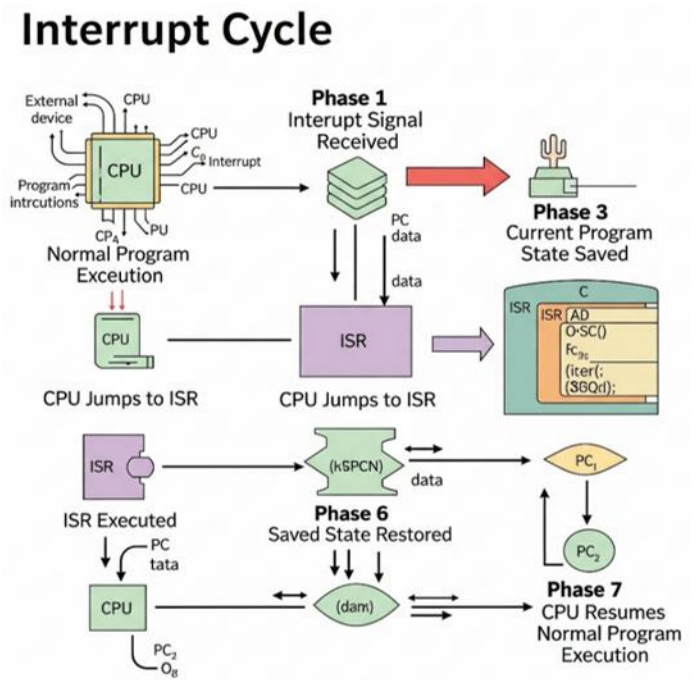
7.วัตถุประสงค์หลักของการเกิด Interrupt:



รูปที่ 4-10 วัตถุประสงค์และความสำคัญของการใช้กลไกการขัดจังหวะในระบบคอมพิวเตอร์

1. **เพิ่มประสิทธิภาพของ CPU (CPU Efficiency):**
   - หากไม่มี Interrupt, CPU จะต้องคอย "เฝ้ารอ" (Polling) ตรวจสอบสถานะของอุปกรณ์ I/O หรือเหตุการณ์ต่างๆ ตลอดเวลา ซึ่งเป็นการสิ้นเปลืองเวลาการประมวลผลอย่างมาก เพราะ CPU จะต้องเสียเวลารออุปกรณ์ I/O ที่ทำงานช้ากว่ามาก ด้วยเหตุนี้การ Interrupt, CPU สามารถทำงานประมวลผลโปรแกรมหลักได้อย่างเต็มที่ และจะหันไปจัดการกับอุปกรณ์ I/O หรือเหตุการณ์อื่น ๆ ก็ต่อเมื่อได้รับสัญญาณ Interrupt เท่านั้น ทำให้ CPU ไม่ต้องรอและสามารถทำงานได้อย่างมีประสิทธิภาพสูงสุด
2. **ตอบสนองต่อเหตุการณ์อย่างทันท่วงที (Timely Response to Events):**
   - Interrupt ช่วยให้ CPU สามารถตอบสนองต่อเหตุการณ์สำคัญที่เกิดขึ้นแบบอะซิงโครนัส (Asynchronous Events) ได้ทันที เช่น การกดปุ่มบนคีย์บอร์ด ซึ่งต้องได้รับการประมวลผลทันทีเพื่อไม่ให้ข้อมูลสูญหาย
3. **จัดการข้อผิดพลาด (Error Handling):**
   - เมื่อเกิดข้อผิดพลาดรุนแรงขึ้น เช่น การหารด้วยศูนย์ หรือการเข้าถึงหน่วยความจำที่ไม่อนุญาต Interrupt จะช่วยให้ระบบสามารถตรวจจับและจัดการกับข้อผิดพลาดเหล่านั้นได้อย่างเหมาะสม อาจเป็นการหยุดโปรแกรมนั้นๆ หรือแจ้งเตือนผู้ใช้
4. **รองรับการทำงานแบบมัลติทาสก์กิ้ง (Multitasking Support):**
   - ระบบปฏิบัติการใช้ Interrupt จาก Timer เพื่อสลับการทำงานระหว่างโปรแกรมต่างๆ ทำให้ผู้ใช้รู้สึกว่าคอมพิวเตอร์กำลังทำงานหลายๆ โปรแกรมพร้อมกันได้

8.กระบวนการ Interrupt Cycle (วัฏจักรอินเทอร์รัปต์) เมื่อ CPU ได้รับสัญญาณ Interrupt จะมีการทำงานเป็นลำดับขั้นตอนดังนี้ :

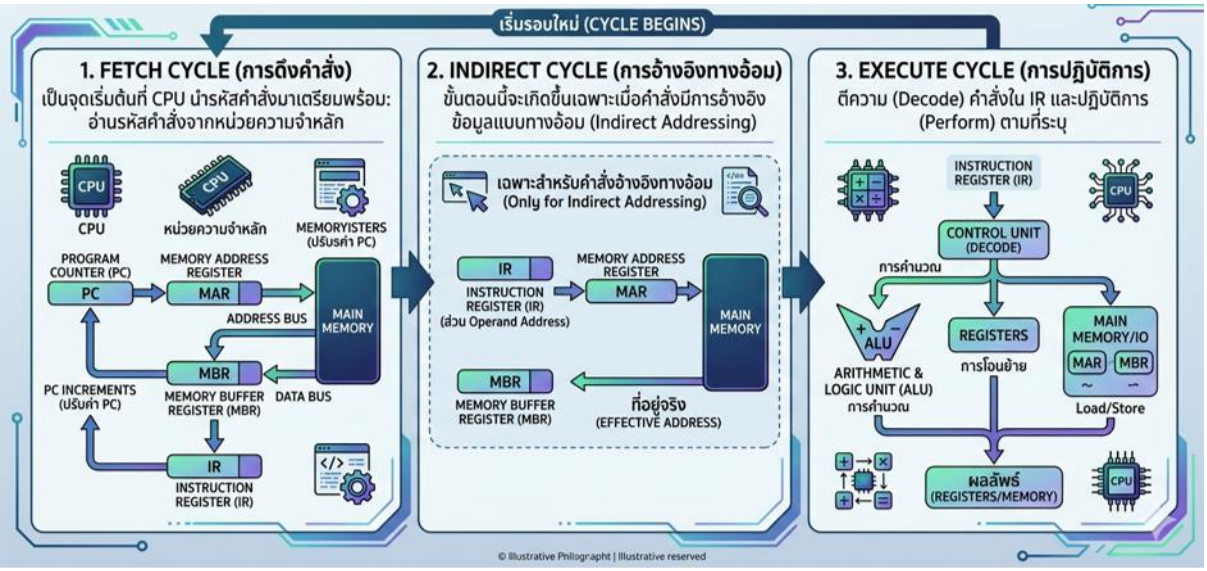


รูปที่ 4-11 ขั้นตอนวงรอบการทำงานของระบบเมื่อเกิดสัญญาณขัดจังหวะ

1. **Normal Program Execution:** CPU กำลังประมวลผลคำสั่งของโปรแกรมหลักตามปกติ
2. **Interrupt Signal Received:** CPU ได้รับสัญญาณ Interrupt จากแหล่งกำเนิดใดๆ (เช่น อุปกรณ์ I/O, Timer, ข้อผิดพลาดในโปรแกรม)
3. **Current Program State Saved:**
   - เมื่อ CPU ตรวจพบ Interrupt และตัดสินใจที่จะตอบสนอง จะหยุดการทำงานของโปรแกรมปัจจุบันชั่วคราว
   - CPU จะบันทึกสถานะปัจจุบันของโปรแกรมไว้บนสแตก (Stack) โดยเฉพาะค่าของ Program Counter (PC) ซึ่งชี้ไปยังที่อยู่ของคำสั่งถัดไปที่ต้องทำหลังจากการจัดการ Interrupt เสร็จสิ้น และอาจรวมถึงค่าในรีจิสเตอร์ที่สำคัญอื่นๆ ด้วย เพื่อให้สามารถกลับมาทำงานเดิมได้อย่างถูกต้อง
4. **CPU Jumps to Interrupt Service Routine (ISR):**
   - หลังจากบันทึกสถานะแล้ว CPU จะกระโดดไปยังตำแหน่งเริ่มต้นของโปรแกรมพิเศษที่เรียกว่า **Interrupt Service Routine (ISR)** หรือ Interrupt Handler
   - ISR คือชุดของคำสั่งที่ถูกออกแบบมาเพื่อจัดการกับ Interrupt นั้นๆ โดยเฉพาะ (เช่น ถ้าเป็น Interrupt จากคีย์บอร์ด ISR ก็จะไปอ่านข้อมูลจากคีย์บอร์ด)
5. **ISR Executed:** CPU ดำเนินการตามคำสั่งใน ISR จนกระทั่งงานที่เกิดจาก Interrupt นั้นได้รับการจัดการเรียบร้อย
6. **Saved State Restored:** เมื่อ ISR ทำงานเสร็จสิ้น CPU จะดึงสถานะของโปรแกรมที่เคยบันทึกไว้ (ค่า PC เดิมและค่ารีจิสเตอร์ต่างๆ) กลับคืนมาจากสแตก
7. **CPU Resumes Normal Program Execution:** CPU กลับไปทำงานตามคำสั่งถัดไปของโปรแกรมหลักที่ถูกขัดจังหวะไปก่อนหน้านี้ได้อย่างถูกต้อง

สรุป การทำงานของ Interrupt Cycle เป็นกลไกพื้นฐานที่สำคัญอย่างยิ่งในการทำให้ระบบคอมพิวเตอร์สมัยใหม่สามารถทำงานได้อย่างราบรื่นและมีประสิทธิภาพ

9.การทำงานของวงรอบคำสั่งของ CPU พร้อมการ Interrupt



รูปที่ 4-12 กลไกการทำงานและการไหลของข้อมูลผ่านรีจิสเตอร์ในวงรอบคำสั่ง (CPU Instruction Cycle and Data Flow)

วงรอบคำสั่งของ CPU (CPU Instruction Cycle)

วงรอบคำสั่งของ CPU คือ กระบวนการพื้นฐานที่ CPU ใช้ในการประมวลผลคำสั่งทีละคำสั่งอย่างต่อเนื่อง ซึ่งเป็นหัวใจสำคัญของการทำงานของคอมพิวเตอร์ แต่ละคำสั่งที่โปรแกรมต้องการให้ CPU ทำงานจะต้องผ่านวัฏจักรนี้เสมอ จนกว่าโปรแกรมจะสิ้นสุดลง หรือระบบจะถูกปิด โดยทั่วไปแล้ว วงรอบคำสั่งของ CPU สามารถแบ่งออกเป็นขั้นตอนหลักๆ ได้แก่:

1. **Fetch Cycle (วงรอบการดึงคำสั่ง):**
    - **หน้าที่:** เป็นขั้นตอนที่ CPU ทำการ **อ่านรหัสคำสั่ง (Instruction)** จากตำแหน่งที่อยู่ (Address) ในหน่วยความจำหลัก
    - **การทำงาน:**
        - ค่าใน **Program Counter (PC)** (ซึ่งเก็บที่อยู่ของคำสั่งถัดไป) จะถูกส่งไปยัง **Memory Address Register (MAR)**
        - MAR จะส่งที่อยู่นี้ผ่าน **Address Bus** ไปยังหน่วยความจำหลัก
        - หน่วยความจำจะส่งข้อมูลคำสั่งที่ตำแหน่งนั้นกลับมาผ่าน **Data Bus** และถูกพักไว้ใน **Memory Buffer Register (MBR)**
        - สุดท้าย ข้อมูลคำสั่งใน MBR จะถูกย้ายไปเก็บไว้ใน **Instruction Register (IR)**
        - หลังจากดึงคำสั่งเสร็จ ค่าใน PC จะถูกปรับให้ชี้ไปยังที่อยู่ของคำสั่งถัดไปที่รออยู่

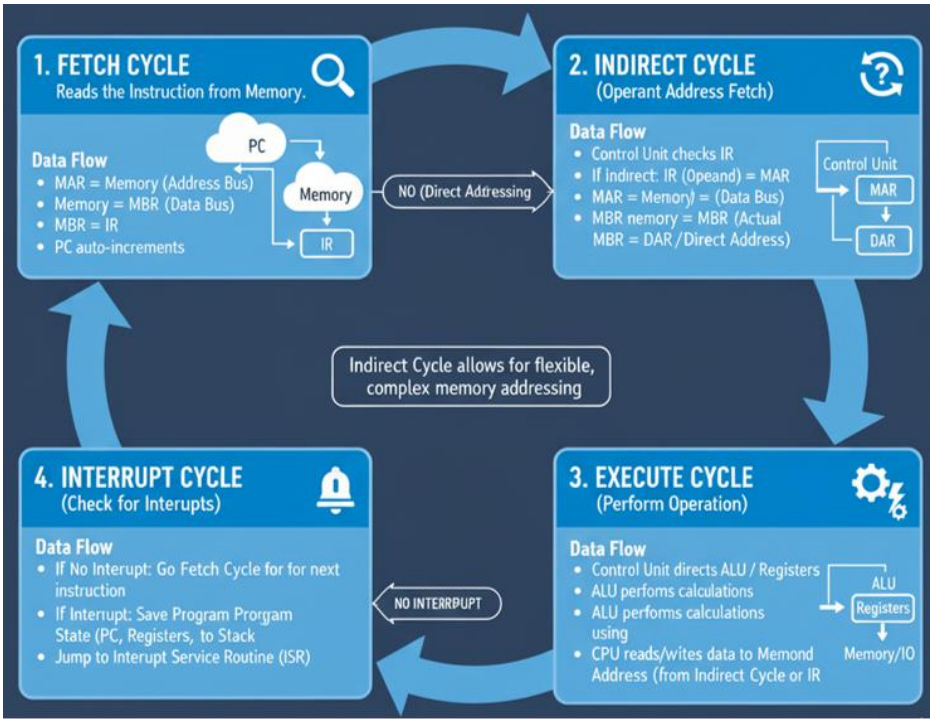
2. **Indirect Cycle (วงรอบการอ้างอิงทางอ้อม) / Indirection:**
    - **หน้าที่:** ขั้นตอนนี้จะเกิดขึ้น **เฉพาะเมื่อคำสั่งที่ดึงมา (จาก Fetch Cycle) มีการอ้างอิงข้อมูลแบบทางอ้อม (Indirect Addressing)** นั่นคือ Operand ของคำสั่งไม่ได้ระบุที่อยู่ของข้อมูลโดยตรง แต่ระบุที่อยู่ของที่อยู่ของข้อมูลอีกทีหนึ่ง
    - **การทำงาน:**
        - Control Unit จะตรวจสอบคำสั่งใน IR หากพบว่าเป็นคำสั่งที่มีการอ้างอิงแบบทางอ้อม ค่า Operand ที่อยู่ใน IR (ซึ่งเป็นที่อยู่ของที่อยู่) จะถูกส่งไปยัง MAR
        - CPU จะทำการอ่านข้อมูลจากหน่วยความจำอีกครั้ง โดยข้อมูลที่อ่านได้ในครั้งนี้คือ **ที่อยู่จริงของ Operand** (ไม่ใช่ตัว Operand เอง)
        - ที่อยู่จริงของ Operand นี้จะถูกนำไปเก็บไว้ใน MBR และพร้อมใช้งานในขั้นตอนต่อไป
3. **Execute Cycle (วงรอบการปฏิบัติการ):**
    - **หน้าที่:** เป็นขั้นตอนที่ CPU **ตีความ (Decode) คำสั่งที่อยู่ใน IR และปฏิบัติการ (Perform the Operation)** ตามที่คำสั่งระบุ
    - **การทำงาน:**
        - หน่วยควบคุม (Control Unit) จะถอดรหัส Opcode และ Operand ของคำสั่ง
        - ถ้าคำสั่งเป็นการคำนวณ (เช่น บวก, ลบ) ข้อมูลจะถูกโหลดเข้าสู่รีจิสเตอร์ที่เกี่ยวข้องและหน่วยคำนวณและตรรกะ (ALU) จะทำการประมวลผล
        - ถ้าคำสั่งเป็นการโอนย้ายข้อมูล (เช่น LOAD, STORE) CPU จะทำการอ่านหรือเขียนข้อมูลจาก/ไปยังหน่วยความจำหรืออุปกรณ์ I/O โดยใช้ MAR และ MBR
        - หลังจากประมวลผลเสร็จสิ้น ผลลัพธ์อาจถูกเก็บไว้ในรีจิสเตอร์ หรือส่งกลับไปยังหน่วยความจำ

สรุปวงรอบคำสั่งของ CPU พร้อมการ Interrupt

*เพื่อเพิ่มประสิทธิภาพและทำให้ CPU สามารถตอบสนองต่อเหตุการณ์ภายนอกได้อย่างทันท่วงที จึงมีการเพิ่ม **Interrupt Cycle** เข้ามาในวงรอบคำสั่งของ CPU โดยทั่วไปแล้ว CPU จะทำการตรวจสอบว่ามี Interrupt เกิดขึ้นหรือไม่ **หลังจากสิ้นสุดการทำงานของ Execute Cycle** ของแต่ละคำสั่ง และก่อนที่จะเริ่ม Fetch คำสั่งถัดไป*

10.กระบวนการ INSTRUCTION CYCLE WITH INDIRECT

วงรอบคำสั่งของ CPU (Instruction Cycle) คือชุดของขั้นตอนที่ CPU ใช้ในการประมวลผลคำสั่งแต่ละคำสั่งอย่างต่อเนื่อง โดยมีวัตถุประสงค์เพื่อให้ CPU สามารถทำงานตามโปรแกรมที่ได้รับมาได้อย่างถูกต้องและมีประสิทธิภาพ ในกรณีที่มีการอ้างอิงข้อมูลแบบทางอ้อม (Indirect Addressing) มีขั้นตอนพิเศษที่เรียกว่า **Indirect Cycle** เพิ่มเข้ามาในวงรอบ



รูปที่ 4-14 แผนภาพสรุปวงรอบการทำงานของคำสั่งและการไหลของข้อมูลผ่านรีจิสเตอร์แบบครบวงจร

1. **Fetch Cycle (วงรอบการดึงคำสั่ง):**
   - **หน้าที่:** เป็นขั้นตอนแรกที่ CPU ทำการ **ดึง (Fetch)** หรืออ่านรหัสคำสั่ง (Instruction) จากหน่วยความจำหลัก
   - **การทำงาน:**
     - ค่าใน **Program Counter (PC)** ซึ่งเก็บที่อยู่ของคำสั่งถัดไปที่จะต้องถูกดึง จะถูกส่งไปยัง **Memory Address Register (MAR)**
     - MAR จะส่งที่อยู่นี้ผ่าน **Address Bus** ไปยังหน่วยความจำ
     - หน่วยความจำจะส่งข้อมูลคำสั่งที่ตำแหน่งนั้นกลับมาผ่าน **Data Bus** และถูกพักไว้ใน **Memory Buffer Register (MBR)** จากนั้น ข้อมูลคำสั่งใน MBR จะถูกย้ายไปเก็บไว้ใน **Instruction Register (IR)** เพื่อรอการถอดรหัสและประมวลผล
     - หลังจากดึงคำสั่งเสร็จ ค่าใน PC จะถูกปรับให้ชี้ไปยังที่อยู่ของคำสั่งถัดไปโดยอัตโนมัติ

2. **Indirect Cycle (วงรอบการอ้างอิงทางอ้อม):**
    - **หน้าที่:** ขั้นตอนนี้จะเกิดขึ้น **เฉพาะเมื่อคำสั่งที่เพิ่งดึงมา (จาก Fetch Cycle) มีการอ้างอิง Operand แบบทางอ้อม (Indirect Addressing)** กล่าวคือ ส่วนของ Operand ในคำสั่งนั้นไม่ได้เป็นข้อมูลโดยตรง หรือที่อยู่ของข้อมูลโดยตรง แต่เป็น **ที่อยู่ของที่อยู่ของข้อมูล (Address of an Address)**
    - **การทำงาน (Data Flow - Indirect Addressing):**
        - หน่วยควบคุม (Control Unit) จะตรวจสอบคำสั่งที่อยู่ใน **Instruction Register (IR)**
        - หากพบว่าคำสั่งนั้นมีการอ้างอิงแบบทางอ้อม ส่วนของ Operand ใน IR (ซึ่งตอนนี้คือที่อยู่ของที่อยู่) จะถูกส่งไปยัง **Memory Address Register (MAR)**
        - CPU จะทำการ **อ่านข้อมูลจากหน่วยความจำอีกครั้ง** โดยใช้ที่อยู่ใน MAR นี้
        - ข้อมูลที่อ่านได้ในครั้งนี้จะถูกส่งกลับมาผ่าน **Data Bus** และถูกพักไว้ใน **Memory Buffer Register (MBR) ข้อมูลที่อยู่ใน MBR ตอนนี้คือ "ที่อยู่จริงของ Operand" (Actual Operand Address)** ที่ CPU ต้องการนำไปใช้ในการประมวลผล
        - ที่อยู่จริงของ Operand นี้จะถูกนำไปใช้ในขั้นตอน Execute ต่อไป (อาจถูกย้ายไปที่รีจิสเตอร์อื่น หรือใช้โดยตรงจาก MBR ขึ้นอยู่กับสถาปัตยกรรม)
3. **Execute Cycle (วงรอบการปฏิบัติการ):**
    - **หน้าที่:** เป็นขั้นตอนที่ CPU **ตีความ (Decode)** คำสั่งที่อยู่ใน **Instruction Register (IR)** และ **ปฏิบัติการ (Perform the Operation)** ตามที่คำสั่งระบุ โดยใช้ Operand ที่ได้จากขั้นตอน Indirect Cycle (หากมี) หรือ Operand โดยตรงจากคำสั่ง
    - **การทำงาน:**
        - หน่วยควบคุมจะสั่งการให้ส่วนประกอบที่เกี่ยวข้อง (เช่น ALU, Registers) ทำงานตามประเภทของคำสั่ง
        - ถ้าเป็นคำสั่งคำนวณ (เช่น บวก, ลบ) ALU จะดำเนินการกับข้อมูล
        - ถ้าเป็นคำสั่งโอนย้ายข้อมูล (เช่น LOAD, STORE) CPU จะทำการอ่านหรือเขียนข้อมูลจาก/ไปยังหน่วยความจำหรืออุปกรณ์ I/O โดยใช้ที่อยู่ของ Operand ที่ได้มา
4. **Interrupt Cycle (วงรอบการตรวจสอบและจัดการ Interrupt):**
    - **หน้าที่:** ขั้นตอนนี้จะเกิดขึ้น **หลังจากสิ้นสุด Execute Cycle** ของแต่ละคำสั่ง เพื่อตรวจสอบว่ามีสัญญาณ Interrupt ค้างอยู่หรือไม่

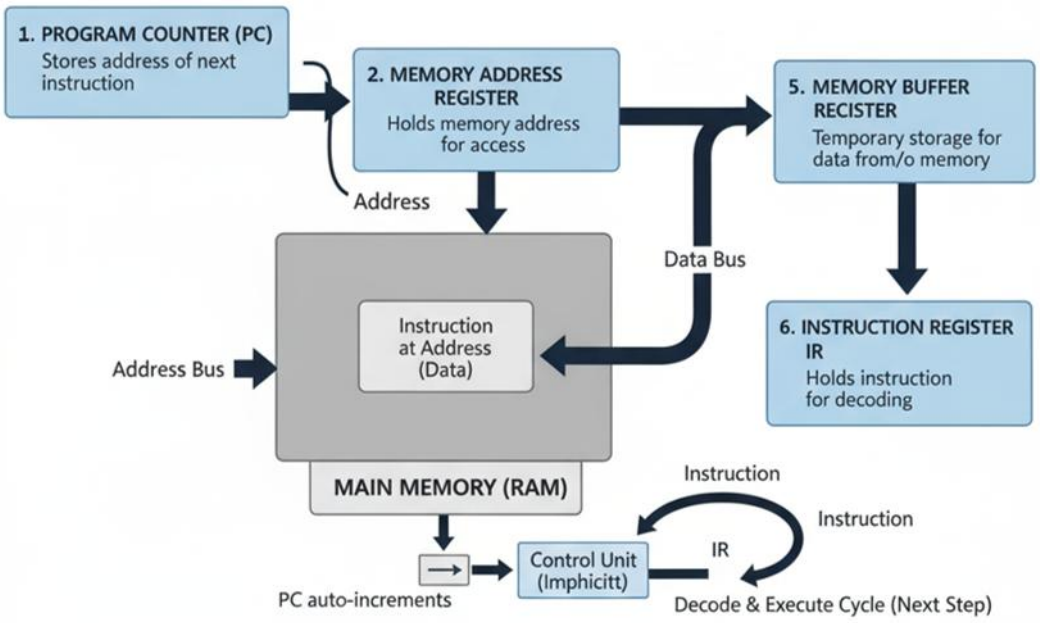
- **การทำงาน:**
  - หากไม่มี Interrupt CPU จะกลับไปที่ **Fetch Cycle** เพื่อดึงคำสั่งถัดไปของโปรแกรมปกติ
  - หากมี Interrupt CPU จะหยุดการทำงานของโปรแกรมหลักชั่วคราว บันทึกสถานะโปรแกรมปัจจุบัน (เช่น PC, Registers) ลงบนสแตก และกระโดดไปทำงานที่ **Interrupt Service Routine (ISR)** เพื่อจัดการกับ Interrupt นั้นๆ เมื่อ ISR ทำงานเสร็จสิ้น CPU จะคืนค่าสถานะและกลับมาทำงานโปรแกรมหลักต่อจากจุดที่ถูกขัดจังหวะ

สรุป: *การรวม Indirect Cycle เข้ามาในวงรอบคำสั่งของ CPU ช่วยให้สถาปัตยกรรมสามารถรองรับการอ้างอิงหน่วยความจำที่ยืดหยุ่นและซับซ้อนมากขึ้น ทำให้โปรแกรมเมอร์สามารถเข้าถึงข้อมูลในรูปแบบต่างๆ ได้อย่างมีประสิทธิภาพ*

11.กระบวนการ DATA FLOW (FETCH DIAGRAM)

การไหลของข้อมูลในวงรอบการดึงคำสั่ง

Fetch Cycle (วงรอบการดึงคำสั่ง) คือขั้นตอนแรกและเป็นพื้นฐานที่สุดในวงรอบคำสั่งของ CPU (Instruction Cycle) มีหน้าที่ในการอ่านรหัสคำสั่ง (Instruction) จากหน่วยความจำหลัก (Main Memory) เพื่อนำมาประมวลผลต่อ การไหลของข้อมูลในขั้นตอนนี้มีความสำคัญอย่างยิ่ง และเกี่ยวข้องกับรีจิสเตอร์และบัสต่างๆ ดังนี้:



รูปที่ 4-15 กลไกการไหลของข้อมูลและเส้นทางการเชื่อมต่อในวงรอบการดึงคำสั่ง

1. **Program Counter (PC) (รีจิสเตอร์ตัวนับโปรแกรม):**
    - **หน้าที่:** PC จะเก็บที่อยู่ (Address) ของคำสั่งถัดไปที่ CPU จะต้องไปดึงมาจากหน่วยความจำ
    - **การไหลของข้อมูล:** ในเริ่มต้นของ Fetch Cycle ค่าที่อยู่ใน PC จะถูกส่งไปยัง **Memory Address Register (MAR)** เพื่อระบุตำแหน่งของคำสั่งที่ต้องการในหน่วยความจำ
2. **Memory Address Register (MAR) (รีจิสเตอร์ที่อยู่หน่วยความจำ):**
    - **หน้าที่:** MAR มีหน้าที่เก็บที่อยู่ของตำแหน่งในหน่วยความจำที่ CPU ต้องการจะเข้าถึง (ในกรณีนี้คือที่อยู่ของคำสั่ง)
    - **การไหลของข้อมูล:** ที่อยู่คำสั่งที่ได้รับจาก PC จะถูกเก็บไว้ใน MAR จากนั้น MAR จะส่งที่อยู่นี้ออกไปบน **Address Bus** เพื่อระบุตำแหน่งในหน่วยความจำหลัก
3. **Address Bus (บัสที่อยู่):**
    - **หน้าที่:** Address Bus เป็นช่องทางที่ใช้สำหรับส่งที่อยู่จาก CPU (โดย MAR) ไปยังหน่วยความจำ เพื่อระบุตำแหน่งของข้อมูลหรือคำสั่งที่ต้องการ
    - **การไหลของข้อมูล:** ที่อยู่คำสั่งจาก MAR จะเดินทางผ่าน Address Bus ไปยัง **Main Memory**
4. **Main Memory (หน่วยความจำหลัก):**
    - **หน้าที่:** หน่วยความจำหลัก (RAM) คือเก็บโปรแกรมและข้อมูลต่างๆ ที่ CPU กำลังใช้งานอยู่
    - **การไหลของข้อมูล:** เมื่อ Main Memory ได้รับที่อยู่จาก Address Bus มันจะค้นหาข้อมูล (ในที่นี้คือรหัสคำสั่ง) ที่ตำแหน่งนั้น และเตรียมส่งกลับไปยัง CPU
5. **Data Bus (บัสข้อมูล):**
    - **หน้าที่:** Data Bus เป็นช่องทางที่ใช้สำหรับส่งข้อมูลไปมาระหว่าง CPU กับหน่วยความจำหลัก หรืออุปกรณ์ I/O อื่นๆ
    - **การไหลของข้อมูล:** รหัสคำสั่งที่ Main Memory ค้นพบ จะถูกส่งกลับมายัง CPU ผ่าน Data Bus และจะถูกพักไว้ใน **Memory Buffer Register (MBR)**
6. **Memory Buffer Register (MBR) (รีจิสเตอร์บัฟเฟอร์หน่วยความจำ):**
    - **หน้าที่:** MBR ทำหน้าที่เป็นบัฟเฟอร์ชั่วคราวสำหรับเก็บข้อมูลที่กำลังถูกโอนย้ายระหว่าง CPU กับหน่วยความจำหลัก (หรืออุปกรณ์ I/O) ไม่ว่าจะเป็นข้อมูลที่อ่านเข้ามาหรือข้อมูลที่จะเขียนออกไป
    - **การไหลของข้อมูล:** รหัสคำสั่งที่เดินทางมาถึง CPU ผ่าน Data Bus จะถูกเก็บไว้ใน MBR ชั่วคราว ก่อนที่จะถูกส่งต่อไปยังจุดหมายสุดท้ายใน Fetch Cycle

7. **Instruction Register (IR) (รีจิสเตอร์คำสั่ง):**
    - **หน้าที่:** IR มีหน้าที่ในการเก็บรหัสคำสั่งที่ CPU เพิ่งดึงมาจากหน่วยความจำ และกำลังจะถูกถอดรหัส (Decode) และประมวลผลโดยหน่วยควบคุม (Control Unit)
    - **การไหลของข้อมูล:** รหัสคำสั่งที่อยู่ใน MBR จะถูกย้ายไปยัง IR ซึ่งเป็นที่เก็บสุดท้ายสำหรับคำสั่งนั้นใน Fetch Cycle เพื่อรอการถอดรหัสและเข้าสู่ขั้นตอนต่อไปคือ Execute Cycle

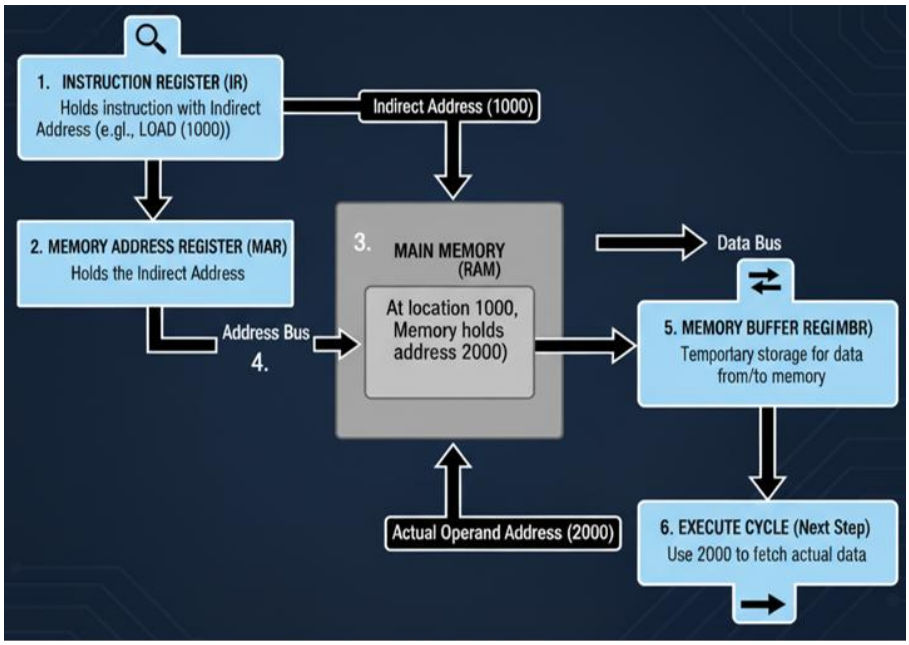
สรุปการไหลของข้อมูลใน Fetch Cycle:

1. *PC ส่งที่อยู่คำสั่งไปยัง MAR*
2. *MAR ส่งที่อยู่ผ่าน Address Bus ไปยัง Main Memory*
3. *Main Memory ค้นหาคำสั่งตามที่อยู่ และส่งคำสั่งนั้นผ่าน Data Bus ไปยัง MBR*
4. *MBR เก็บคำสั่งชั่วคราว และส่งต่อคำสั่งไปยัง IR*

โดยที่หลังจากที่คำสั่งถูกโหลดเข้าสู่ IR แล้ว CPU ก็จะพร้อมที่จะเข้าสู่ขั้นตอนถัดไปในวงรอบคำสั่ง ซึ่งอาจจะเป็น Indirect Cycle (หากคำสั่งมีการอ้างอิงทางอ้อม) หรือ Execute Cycle ทันที

12.DATA FLOW (INDIRECT ADDRESSING): การไหลของข้อมูลในวงรอบการอ้างอิงทางอ้อม

Indirect Cycle (วงรอบการอ้างอิงทางอ้อม) เป็นขั้นตอนที่เกิดขึ้นในวงรอบคำสั่งของ CPU **เฉพาะเมื่อคำสั่งที่ CPU เพิ่งดึงมาใน Fetch Cycle มีการอ้างอิงข้อมูลแบบทางอ้อม** (Indirect Addressing) ซึ่งหมายความว่า ส่วนของ Operand ในคำสั่งนั้นไม่ได้ระบุข้อมูลโดยตรง หรือที่อยู่ของข้อมูลโดยตรง แต่ระบุ **ที่อยู่ของที่อยู่ของข้อมูล (Address of an Address)** ที่แท้จริง



รูปที่ 4-16 กลไกการไหลของข้อมูลในวงรอบการอ้างอิงทางอ้อมเพื่อค้นหาที่อยู่จริงของข้อมูล

วัตถุประสงค์ของ Indirect Cycle คือการค้นหา **ที่อยู่จริงของ Operand (Actual Operand Address)** ที่ CPU ต้องการนำไปใช้ในการประมวลผลในขั้นตอน Execute Cycle ถัดไป

1. **Instruction Register (IR) (รีจิสเตอร์คำสั่ง):**
    - **สถานะเริ่มต้น:** ในขั้นตอนนี้ คำสั่งที่ CPU ดึงมาใน Fetch Cycle จะถูกเก็บอยู่ใน IR แล้ว โดยส่วนของ Operand ของคำสั่งนี้จะบรรจุ **ที่อยู่แบบทางอ้อม** (เช่น สมมติว่าคำสั่งคือ LOAD (1000) หมายถึงให้โหลดข้อมูลจากที่อยู่ที่ 1000 แต่ที่อยู่ 1000 นี้ไม่ได้เก็บข้อมูลโดยตรง แต่เก็บที่อยู่จริงของข้อมูลอีกที)
    - **การไหลของข้อมูล:** หน่วยควบคุม (Control Unit) จะถอดรหัสคำสั่งใน IR และตรวจพบว่าเป็นการอ้างอิงแบบทางอ้อม จากนั้น **ที่อยู่แบบทางอ้อม** ที่อยู่ในส่วน Operand ของ IR (ในตัวอย่างคือ 1000) จะถูกส่งไปยัง **Memory Address Register (MAR)**
2. **Memory Address Register (MAR) (รีจิสเตอร์ที่อยู่หน่วยความจำ):**
    - **หน้าที่:** MAR มีหน้าที่เก็บที่อยู่ของตำแหน่งในหน่วยความจำที่ CPU ต้องการจะเข้าถึง
    - **การไหลของข้อมูล:** ที่อยู่แบบทางอ้อม (1000) ที่ได้รับจาก IR จะถูกเก็บไว้ใน MAR จากนั้น MAR จะส่งที่อยู่นี้ออกไปบน **Address Bus** เพื่อระบุตำแหน่งในหน่วยความจำหลัก
3. **Address Bus (บัสที่อยู่):**
    - **หน้าที่:** Address Bus เป็นช่องทางที่ใช้สำหรับส่งที่อยู่จาก CPU (โดย MAR) ไปยังหน่วยความจำ
    - **การไหลของข้อมูล:** ที่อยู่แบบทางอ้อม (1000) จาก MAR จะเดินทางผ่าน Address Bus ไปยัง **Main Memory**
4. **Main Memory (หน่วยความจำหลัก):**
    - **หน้าที่:** หน่วยความจำหลัก (RAM) คือที่เก็บข้อมูลและคำสั่ง
    - **การไหลของข้อมูล:** เมื่อ Main Memory ได้รับที่อยู่ (1000) จาก Address Bus มันจะค้นหาข้อมูลที่ตำแหน่งนั้น ในกรณีของการอ้างอิงทางอ้อม ข้อมูลที่อยู่ที่ 1000 นี้ **ไม่ใช่ตัว Operand เอง** แต่เป็น **ที่อยู่จริงของ Operand (Actual Operand Address)** (เช่น สมมติว่าที่อยู่ 1000 เก็บค่า 2000 ซึ่ง 2000 คือที่อยู่จริงของข้อมูลที่เราต้องการ) Main Memory จะส่งค่า 2000 นี้กลับมายัง CPU
5. **Data Bus (บัสข้อมูล):**
    - **หน้าที่:** Data Bus เป็นช่องทางที่ใช้สำหรับส่งข้อมูลไปมาระหว่าง CPU กับหน่วยความจำหลัก

- **การไหลของข้อมูล: ที่อยู่จริงของ Operand (2000)** ที่อ่านได้จาก Main Memory จะถูกส่งกลับมายัง CPU ผ่าน Data Bus และจะถูกพักไว้ใน **Memory Buffer Register (MBR)**

6. **Memory Buffer Register (MBR) (รีจิสเตอร์บัฟเฟอร์หน่วยความจำ):**
   - **หน้าที่:** MBR ทำหน้าที่เป็นบัฟเฟอร์ชั่วคราวสำหรับเก็บข้อมูลที่กำลังถูกโอนย้ายระหว่าง CPU กับหน่วยความจำหลัก
   - **การไหลของข้อมูล: ที่อยู่จริงของ Operand (2000)** ที่เดินทางมาถึง CPU ผ่าน Data Bus จะถูกเก็บไว้ใน MBR ชั่วคราว

สรุปการไหลของข้อมูลใน Indirect Cycle:

- *IR (มีที่อยู่ทางอ้อม) -> MAR (เก็บที่อยู่ทางอ้อม) -> Address Bus -> Main Memory (ค้นหาที่อยู่ทางอ้อม) -> Data Bus (ส่งที่อยู่จริงของ Operand) -> MBR (เก็บที่อยู่จริงของ Operand)*

*หลังจากสิ้นสุด Indirect Cycle แล้ว **MBR จะบรรจุ "ที่อยู่จริงของ Operand"** ซึ่ง CPU จะนำไปใช้ในขั้นตอน **Execute Cycle** ถัดไป เพื่อเข้าถึงตัว Operand จริงๆ ในหน่วยความจำและดำเนินการตามคำสั่ง*

ชุดข้อสอบเพื่อทบทวนความเข้าใจบทที่ 4 ไมโครโปรเซสเซอร์และการประมวลผล (40 ข้อ)

คำชี้แจง: ให้นักศึกษาเลือกคำตอบที่ถูกต้องที่สุดเพียงข้อเดียว

1.สถาปัตยกรรมคอมพิวเตอร์แบบ CISC (Complex Instruction Set Computer) มีลักษณะเด่นอย่างไร ?

ก. ชุดคำสั่งมีความยาวเท่ากันทั้งหมดเพื่อความรวดเร็วในการประมวลผล

ข. ใช้รีจิสเตอร์จำนวนมาก และเน้นการทำงานด้วยฮาร์ดแวร์

ค. ชุดคำสั่งซับซ้อน แต่ละคำสั่งมีขั้นตอนการทำงานและขนาดของรหัสคำสั่งที่ไม่เท่ากัน

ง. เน้นการทำงานแบบ Pipeline ได้อย่างมีประสิทธิภาพสูงสุดโดยไม่มีปัญหา Hazard

เฉลย: ค

2.ข้อใดคือจุดเด่นหลักสถาปัตยกรรม RISC (Reduced Instruction Set Computer) เมื่อเทียบกับ CISC ?

ก. ชุดคำสั่งมีความหลากหลายและซับซ้อน ทำให้โปรแกรมเมอร์เขียนโปรแกรมได้ง่าย

ข. ชุดคำสั่งของไมโครโปรเซสเซอร์มีความยาวของรหัสคำสั่งเท่ากันทั้งหมด

ค. ใช้หน่วยความจำในการจัดเก็บชุดคำสั่งจำนวนมากและใช้เวลาในการประมวลผลแต่ละคำสั่งนาน

ง. ตัวอย่างเช่น ไมโครโปรเซสเซอร์ตระกูล Intel x86-64bit

เฉลย: ข

3.การเปรียบเทียบสถาปัตยกรรม CISC และ RISC ข้อใดถูกต้องที่สุด ?

ก. CISC เน้นการทำงานแบบ Hardware Control ส่วน RISC เน้น Microcode

ข. CISC มีชุดคำสั่งจำนวนมากและซับซ้อน ส่วน RISC มีชุดคำสั่งจำนวนน้อยและเรียบง่าย

ค. RISC มีความเสี่ยงที่จะเกิดปัญหา Pipeline Hazard สูงกว่า CISC

ง. ไมโครโปรเซสเซอร์ของ Apple ตระกูล M-series (ARM) ใช้สถาปัตยกรรมแบบ CISC

เฉลย: ข

4.คำสั่งของ CISC มีความยาวไม่เท่ากัน ทำให้เกิดปัญหาในการทำงานแบบไปป์ไลน์ (Pipelining) อย่างไร ?

ก. คำสั่งซับซ้อนเกินไปทำให้ ALU ประมวลผลผิดพลาด

ข. ความยาวของคำสั่งที่ไม่เท่ากันทำให้เกิดปัญหาในการทำงานแบบไปป์ไลน์

ค. ทำให้ Control Unit ต้องทำงานซ้ำซ้อน

ง. คำสั่งใช้หน่วยความจำในการจัดเก็บข้อมูลมากเกินไป

เฉลย: ข

5.ไมโครโปรเซสเซอร์ในปัจจุบันที่ใช้สถาปัตยกรรม CISC และเป็นที่นิยมคือตระกูลใด?

ก. Qualcomm Snapdragon

ข. Intel ตระกูล x86-64bit

ค. ARM Cortex-A

ง. PowerPC

เฉลย: ข

6.ส่วนประกอบโครงสร้างพื้นฐานภายในของไมโครโปรเซสเซอร์ประกอบด้วยอะไรบ้าง?

ก. Main Memory, I/O Modules, System Bus

ข. Control Unit, ALU, Cache Memory

ค. ALU, Registers, Internal CPU Interconnection, Control Unit

ง. Instruction Register, Program Counter, Memory Address Register

เฉลย: ค

7.หน้าที่หลักของ Register ภายใน CPU คืออะไร?

ก. เก็บชุดคำสั่งและข้อมูลขนาดใหญ่สำหรับโปรแกรมทั้งหมด

ข. เป็นหน่วยความจำหลักที่ CPU ใช้ติดต่อโดยตรง

ค. ทำหน้าที่เป็นบัฟเฟอร์ในการโอนย้ายข้อมูลระหว่าง CPU กับ I/O

ง. เป็นหน่วยเก็บข้อมูลชั่วคราวที่มีความเร็วในการเข้าถึงข้อมูลสูงที่สุดในระบบ

เฉลย: ง

8.General Purpose Register (รีจิสเตอร์อเนกประสงค์) มีหน้าที่หลักตามชื่อเรียกคืออะไร?

ก. ใช้เก็บข้อมูลที่กำลังถูกดำเนินการ หรือผลลัพธ์จากการคำนวณ

ข. ใช้เก็บแอดเดรสของคำสั่งถัดไปที่ต้อง Fetch

ค. ใช้สำหรับเก็บที่อยู่เริ่มต้นของเซกเมนต์หน่วยความจำ

ง. ใช้สำหรับการเก็บค่าตัวชี้สแตก (Stack Pointer)

เฉลย: ก

9.รีจิสเตอร์ประเภท User-Visible Registers แบบ Data Register มีหน้าที่ใด?

ก. ใช้เก็บที่อยู่ของหน่วยความจำที่กำลังจะเข้าถึง

ข. ใช้ชี้ไปยังตำแหน่งเริ่มต้นของสแตกในหน่วยความจำ

ค. ใช้สำหรับคำนวณที่อยู่จริง (Physical Address) ในหน่วยความจำ

ง. ใช้เก็บข้อมูลที่ต้องการนำมาประมวลผล

เฉลย: ง

10.Address Registers (รีจิสเตอร์ที่อยู่) มีวัตถุประสงค์หลักในการใช้งานคืออะไร?

ก. เก็บรหัสคำสั่ง (Opcode) ที่ถูกดึงเข้ามาจากหน่วยความจำ

ข. ใช้เก็บที่อยู่ของหน่วยความจำที่ CPU กำลังจะเข้าถึง

ค. ใช้เป็นตัวควบคุมการทำงานของ ALU

ง. ใช้เก็บสถานะการทำงานล่าสุดของ CPU

เฉลย: ข

11.Index Registers (รีจิสเตอร์ดัชนี) มักถูกนำไปใช้ในการกำหนดแอดเดรสแบบใด?

ก. Direct Addressing

ข. Indirect Addressing

ค. Register Addressing

ง. Indexed Addressing

เฉลย: ง

12.ในระบบคอมพิวเตอร์ที่ใช้การแบ่งส่วน (Segmentation) Segment Registers มีบทบาทสำคัญอย่างไร?

ก. เก็บค่า Offset ของตำแหน่งข้อมูลภายในเซกเมนต์

ข. เก็บที่อยู่เริ่มต้นของแต่ละเซกเมนต์ในหน่วยความจำ

ค. ใช้เป็นดัชนีในการเข้าถึงข้อมูลแบบ Array

ง. ใช้เป็นตัวนับรอบการทำงานของ CPU

เฉลย: ข

13.Stack Pointer (SP) หรือรีจิสเตอร์ตัวชี้สแตกมีหน้าที่หลักในการทำงานกับสแตกอย่างไร?

ก. เก็บขนาดทั้งหมดของหน่วยความจำสแตก

ข. ชี้ไปยังตำแหน่งด้านบนสุด (Top) ของสแตก

ค. ใช้สำหรับคำนวณ Effective Address

ง. ใช้เก็บค่าที่ถูกดึงออกมาจากสแตก

เฉลย: ข

14.Flags Register หรือ Condition Codes Register ใช้ในการจัดเก็บข้อมูลประเภทใดเป็นหลัก?

ก. ข้อมูลตัวเลขที่ได้จากการคำนวณ

ข. ที่อยู่ของข้อมูลในหน่วยความจำแคช

ค. สถานะล่าสุดที่ได้จากการประมวลผลของ ALU

ง. ค่าคงที่ที่ใช้ในคำสั่ง

เฉลย: ค

15.แฟล็ก (Flag) ในรีจิสเตอร์รหัสเงื่อนไข / แฟล็ก ที่ระบุว่าผลลัพธ์จากการดำเนินการทางคณิตศาสตร์มีค่าเท่ากับศูนย์คือแฟล็กใด?

ก. Overflow Flag

ข. Sign Flag

ค. Carry Flag

ง. Zero Flag

เฉลย: ง

16.ในระหว่างการคำนวณทางคณิตศาสตร์ หากผลลัพธ์มีค่าเกินกว่าความจุของรีจิสเตอร์ แฟล็กใดใน Condition Codes Register จะถูกตั้งค่าเพื่อบ่งบอกถึงเหตุการณ์นี้?

ก. Sign Flag ข. Overflow Flag

ค. Parity Flag ง. Interrupt Flag

เฉลย: ข

17.Program Counter (PC) มีหน้าที่ควบคุมการไหลของโปรแกรมอย่างไร?

ก. เก็บคำสั่งที่กำลังถูกประมวลผลในปัจจุบัน

ข. เก็บที่อยู่ของข้อมูลที่กำลังจะถูกโอนย้าย

ค. เก็บที่อยู่ของคำสั่งถัดไปที่ CPU จะต้องดึง (Fetch) เข้ามา

ง. เก็บสถานะทั้งหมดของ CPU

เฉลย: ค

18.Instruction Register (IR) มีความสำคัญในวงรอบคำสั่งอย่างไร?

ก. เก็บที่อยู่ของคำสั่งถัดไป

ข. เก็บรหัสคำสั่งปัจจุบันที่ถูกดึงเข้ามาจากหน่วยความจำ

ค. ทำหน้าที่เป็นบัฟเฟอร์สำหรับโอนย้ายข้อมูลกับหน่วยความจำ

ง. เก็บสถานะการทำงานของ CPU

เฉลย: ข

19.Memory Address Register (MAR) ทำงานร่วมกับ System Bus อย่างไร?

ก. เก็บข้อมูลที่กำลังถูกอ่านหรือเขียน

ข. เก็บที่อยู่ของหน่วยความจำที่จะถูกส่งไปยัง Address Bus

ค. เก็บคำสั่งที่ถูก Decode แล้ว

ง. เก็บสถานะของโปรแกรมทั้งหมด

เฉลย: ข

20.Memory Buffer Register (MBR) มีบทบาทในการสื่อสารข้อมูลระหว่าง CPU กับหน่วยความจำอย่างไร?

ก. เก็บรหัสคำสั่ง (Opcode) ก่อนที่จะถูกส่งไปยัง Control Unit

ข. เป็นบัฟเฟอร์ชั่วคราวสำหรับเก็บข้อมูล/คำสั่งที่โอนย้ายระหว่าง CPU กับหน่วยความจำหลัก

ค. ใช้เก็บที่อยู่ของหน่วยความจำที่ถูกต้อง

ง. ใช้ในการจัดการการเข้าถึงหน่วยความจำแคช

เฉลย: ข

21.Program Status Word (PSW) มีหน้าที่แตกต่างจาก Control and Status Registers อื่น ๆ อย่างไร?

ก. ใช้เก็บสถานะการทำงานและข้อมูลควบคุมทั้งหมดของโปรแกรม

ข. ใช้เก็บค่าแอดเดรสสำหรับการอ้างอิงทางอ้อมเท่านั้น

ค. ใช้เก็บผลลัพธ์สุดท้ายของการคำนวณ

ง. ใช้เก็บค่ารีจิสเตอร์ทั้งหมดเพื่อทำ Context Switching

เฉลย: ก

22.ในการบริหารจัดการองค์ประกอบของ Registers หากเกิดเหตุการณ์ Interrupt ข้อมูลใดที่จำเป็นต้องถูกบันทึกไว้ก่อนเพื่อกลับมาทำงานเดิมได้ถูกต้อง?

ก. ค่าใน Memory Buffer Register (MBR)

ข. ค่าใน Instruction Register (IR)

ค. ค่าใน Program Counter (PC) และ Program Status Word (PSW)

ง. ค่าใน Memory Address Register (MAR)

เฉลย: ค

23.วงรอบคำสั่ง (Instruction Cycle) ของ CPU ประกอบด้วยขั้นตอนพื้นฐานใดบ้าง?

ก. Decode และ Execute

ข. Fetch และ Execute

ค. Interrupt และ Indirect

ง. Memory Read และ Memory Write

เฉลย: ข

24.ในการทำงานปกติของ CPU นั้น วงรอบคำสั่ง (Instruction Cycle) จะวนซ้ำไปเรื่อย ๆ จนกระทั่งเกิดเงื่อนไขใด?

ก. โปรแกรมมีการเรียกใช้ฟังก์ชันย่อย ข. มีคำสั่งที่ต้องการเข้าถึง I/O

ค. คำสั่งให้หยุดการทำงาน (Halt) หรือปิดเครื่อง ง. มีการโอนย้ายข้อมูลผ่าน Data Bus

เฉลย: ค

25.วัตถุประสงค์หลักของการเกิด Interrupt คืออะไร?

ก. เพื่อให้ CPU หยุดทำงานทันทีเมื่อพบข้อผิดพลาด

ข. เพื่อเพิ่มความเร็วในการประมวลผลของ ALU

ค. เพื่อให้ CPU สามารถตอบสนองต่อเหตุการณ์ภายนอก/ภายในที่สำคัญได้ทันท่วงที

ง. เพื่อทำการสลับการทำงานระหว่าง CISC และ RISC

เฉลย: ค

26.ขั้นตอนใดที่เกิดขึ้นในวงรอบคำสั่ง (Instruction Cycle) เพื่อตรวจสอบและตอบสนองต่อสัญญาณอินเทอร์รัปต์?

ก. Execute Cycle
ข. Indirect Cycle
ค. Fetch Cycle
ง. Interrupt Cycle

เฉลย: ง

27.กระบวนการ Interrupt Cycle (วัฏจักรอินเทอร์รัปต์) เริ่มต้นด้วยการกระทำใด?

ก. CPU ตรวจสอบคำสั่งใน IR ว่ามี Interrupt flag ถูกตั้งหรือไม่
ข. CPU บันทึกค่าที่เก็บอยู่ใน Program Counter (PC) ไว้ก่อน
ค. CPU กระโดดไปประมวลผลคำสั่งถัดไปที่ระบุใน PC ทันที
ง. CPU ทำการประมวลผลคำสั่งที่ถูก Interrupt เข้ามา

เฉลย: ข

28.ในกระบวนการ Interrupt Cycle ค่าใน PC จะถูกโอนย้ายไปเก็บไว้ที่ใดก่อนที่จะเขียนลงหน่วยความจำหลัก?

ก. Instruction Register (IR)
ข. Memory Buffer Register (MBR)
ค. Program Status Word (PSW)
ง. Address Bus

เฉลย: ข

29.วงรอบคำสั่งของ CPU พร้อมแทรกการทำงานของ Interrupt มีขั้นตอนการทำงานใดเพิ่มเติมจากวงรอบคำสั่งปกติ?

ก. หลังจาก Execute Cycle เสร็จขั้นตอนการตรวจสอบ Interrupt ก่อนเริ่ม Fetch Cycle ถัดไป
ข. Interrupt Cycle จะเกิดขึ้นก่อน Fetch Cycle เสมอ
ค. Interrupt Cycle จะเกิดขึ้นแทนที่ Execute Cycle
ง. Fetch Cycle จะถูกยกเลิกเมื่อมี Interrupt เกิดขึ้น

เฉลย: ก

30.หากมีสัญญาณอินเทอร์รัปต์เข้ามาในขณะที่ CPU กำลังประมวลผลคำสั่งอยู่ CPU จะตอบสนองในขั้นตอนใด?

ก. ในทันทีที่รับสัญญาณ
ข. เมื่อสิ้นสุดวงรอบคำสั่งปัจจุบัน
ค. เมื่อทำการ Operand Fetch เสร็จสิ้น
ง. เมื่อคำสั่งใน IR เป็นคำสั่ง I/O

เฉลย: ข

31.วงรอบคำสั่งที่เกิดจากการอ้างแอดเดรสแบบทางอ้อม (Indirect Addressing) เรียกว่าอะไร?

ก. Execute Cycle ข. Interrupt Cycle

ค. Indirect Cycle ง. Data Flow Cycle

เฉลย: ค

32.เหตุผลหลักที่ต้องมี INSTRUCTION CYCLE WITH INDIRECT คืออะไร?

ก. เพื่อจัดการกับคำสั่งที่มีขนาดใหญ่เกินกว่าจะเก็บใน IR ได้

ข. เพื่อจัดการกับการอ้างถึงตำแหน่งของข้อมูลที่ไม่ได้เป็นแอดเดรสที่แท้จริง (Actual Operand Address)

ค. เพื่อให้สามารถกระโดดไปยังตำแหน่งคำสั่งถัดไปได้

ง. เพื่อให้ CPU สามารถดึงข้อมูลจากที่อยู่จริงที่ถูกเก็บไว้ในที่อยู่แรกที่คำสั่งระบุได้

เฉลย: ง

33.ในกระบวนการ DATA FLOW (FETCH DIAGRAM) ข้อมูลที่ถูกอ่านจากหน่วยความจำหลัก (Main Memory) จะถูกพักไว้ที่รีจิสเตอร์ใดก่อนจะถูกส่งไปยัง Instruction Register (IR)?

ก. Program Counter (PC) ข. Memory Address Register (MAR)

ค. Memory Buffer Register (MBR) ง. Accumulator (AC)

เฉลย: ค

34.ตามกระบวนการ DATA FLOW (FETCH DIAGRAM) ค่าใน Program Counter (PC) จะถูกโอนย้ายไปที่ Memory Address Register (MAR) เพื่อวัตถุประสงค์ใด?

ก. เพื่อเพิ่มค่า PC สำหรับคำสั่งถัดไป

ข. เพื่อเก็บคำสั่งที่ถูก Fetch มา

ค. เพื่อระบุตำแหน่งของคำสั่งที่ต้องการดึงเข้ามาจากหน่วยความจำ

ง. เพื่อใช้เป็นแอดเดรสสำหรับ Operand

เฉลย: ค

35.ในกระบวนการ DATA FLOW (INDIRECT DIAGRAM) เมื่อสิ้นสุดกระบวนการ Fetch หน่วยควบคุมจะตรวจสอบข้อมูลใน IR และหากมีการอ้างอิงแบบ Indirect Addressing ข้อมูลแอดเดรสทางอ้อมที่อยู่ใน MBR จะถูกโอนย้ายไปยังรีจิสเตอร์ใดเพื่อเริ่ม Indirect Cycle?

ก. Instruction Register (IR) ข. Program Counter (PC)

ค. Memory Address Register (MAR) ง. Accumulator (AC)

เฉลย: ค

36.ในกระบวนการ DATA FLOW (INDIRECT DIAGRAM) หลังจากที่ MBR ได้รับที่อยู่จริงของ Operand แล้ว ค่าที่อยู่ใน MBR จะถูกโอนย้ายไปที่ใดเพื่อดำเนินการต่อ?

ก. MAR เพื่อเตรียมดึง Operand จริง หรือ IR เพื่อปรับปรุงคำสั่ง

ข. PC เพื่อเตรียม Fetch คำสั่งถัดไป

ค. ALU เพื่อเริ่มการคำนวณ

ง. Control Unit เพื่อตรวจสอบสถานะ

เฉลย: ก

37.ในการทำงานของ CPU Instruction Cycle หากมีการอ้างถึง Operand ในหน่วยความจำโดยใช้ที่อยู่แบบทางอ้อม (Indirect Addressing) จะมีการแทรกขั้นตอนใดเข้ามาในวงรอบคำสั่ง?

ก. Execute Cycle

ข. Indirect Cycle

ค. Interrupt Cycle

ง. Register Update Cycle

เฉลย: ข

38.ข้อใดคือคำสั่งที่ถูกต้องตามวงรอบคำสั่งของ CPU พร้อมแทรกการทำงานของ Interrupt?

ก. Fetch $\rightarrow$ Interrupt $\rightarrow$ Execute

ข. Fetch $\rightarrow$ Indirect $\rightarrow$ Interrupt $\rightarrow$ Execute

ค. Execute $\rightarrow$ Interrupt Check $\rightarrow$ Fetch

ง. Execute $\rightarrow$ Indirect $\rightarrow$ Interrupt $\rightarrow$ Fetch

เฉลย: ค

39.ข้อใดกล่าวถึงโครงสร้างสถาปัตยกรรมคอมพิวเตอร์แบบ RISC ได้ถูกต้องที่สุด?

ก. มีชุดคำสั่งน้อยและเรียบง่าย ทำให้สามารถทำงานแบบ Pipeline ได้อย่างมีประสิทธิภาพ

ข. คำสั่งแต่ละคำสั่งใช้เวลาในการประมวลผลมาก จึงต้องใช้หน่วยความจำมาก

ค. ตัวอย่างไมโครโปรเซสเซอร์คือตระกูล Intel Pentium

ง. การอ้างแอดเดรสได้หลายแบบ เพื่อให้โปรแกรมเมอร์เขียนโปรแกรมได้ง่าย

เฉลย: ก

40.ในวงรอบคำสั่งปกติของ CPU การตีความคำสั่งเพื่อปฏิบัติงานนั้นเกิดขึ้นในระยะใด?

ก. Fetch

ข. Execute

ค. Indirect

ง. Interrupt

เฉลย: ข